

Laporan Tugas Besar 1
IF3270 Machine Learning



13522029 - Ignatius Jhon Hezekiel Chan
13522043 - Daniel Mulia Putra Manurung
13522093 - Matthew Vladimir Hutabarat

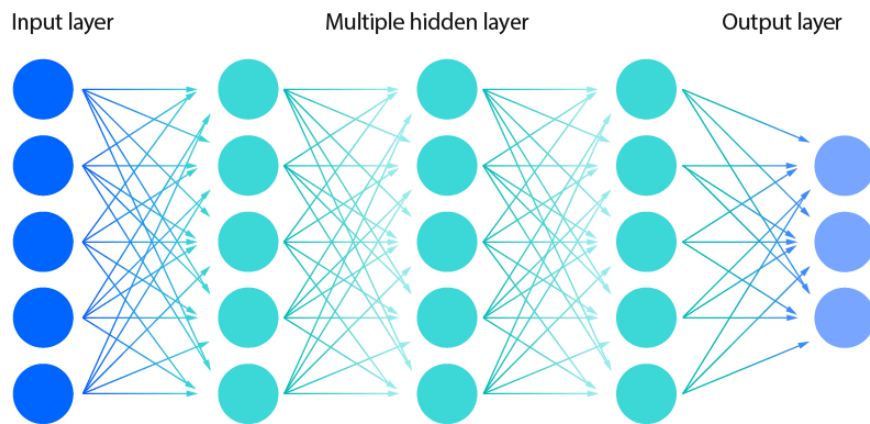
PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

Daftar Isi

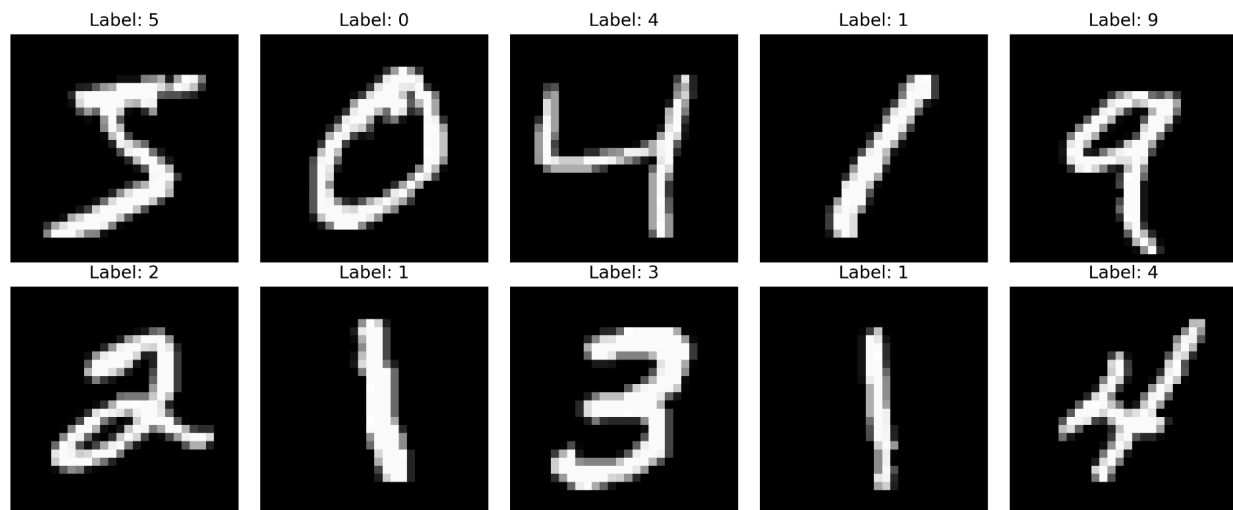
Daftar Isi	2
Deskripsi Persoalan	3
Pembahasan	8
2.1 Penjelasan Implementasi	8
2.1.1 Deskripsi Kelas	8
2.1.1.1 Class Activation	8
2.1.1.2 Class Losses	9
2.1.1.3 Class Initializers	12
2.1.1.4 Class Regularizer	12
2.1.1.5 Class RMSNorm	13
2.1.1.6 Class FFNN	14
2.1.2 Forward Propagation	24
2.1.3 Backward Propagation	25
2.1.4 Update Weight	28
2.2 Pengujian	29
2.2.1 Pengaruh Depth dan Width	29
2.2.2 Pengaruh Fungsi Aktivasi	34
2.2.3 Pengaruh Learning Rate	38
2.2.4 Pengaruh Inisialisasi Bobot	41
2.2.5 Pengaruh Regularisasi (bonus)	45
2.2.6 Pengaruh Normalisasi RMSNorm (bonus)	48
2.2.7 Perbandingan dengan library sklearn	50
Kesimpulan dan Saran	53
Pembagian Tugas	54
Referensi	55

Deskripsi Persoalan

Pada tugas ini, kami sebagai peserta kuliah IF3270 Pembelajaran Mesin ditugaskan untuk membuat salah satu tipe dari Neural Network dari *scratch*, yakni **Feedforward Neural Network (FFNN)**. Model ini kemudian akan dilakukan testing dengan dataset OpenMP MNIST_784 yang berisi gambar digit angka dari 0 - 9, dengan masing-masing gambar memiliki resolusi 28x28. FFNN ini kemudian akan dilakukan testing untuk memprediksi digit dari suatu gambar input.



Gambar Feedforward Neural Network



Gambar Dataset OpenMP MNIST_784

Model FFNN yang akan diimplementasikan harus memenuhi beberapa ketentuan terkait implementasi fungsi aktivasi, fungsi *Loss*, metode Inisialisasi, metode regularisasi, dan metode normalisasi. Ketentuan-ketentuan tersebut adalah sebagai berikut.

- FFNN yang diimplementasikan dapat menerima jumlah neuron dari tiap layer (termasuk input layer dan output layer)

- FFNN yang diimplementasikan dapat menerima fungsi aktivasi dari tiap layer. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

- FFNN yang diimplementasikan dapat menerima fungsi loss dari model tersebut. Pilihan loss function yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Loss	Definisi Fungsi
MSE	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Binary Cross-Entropy	$\mathcal{L}_{BCE} = -\frac{1}{n} \sum_{i=1}^n (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
Categorical Cross-Entropy	$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C (y_{ij} \log \hat{y}_{ij})$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size

- Catatan:
 - Binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2
 - Log yang digunakan merupakan logaritma natural (logaritma dengan basis e)
- Terdapat mekanisme untuk inisialisasi bobot tiap neuron (termasuk bias). Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah sebagai berikut:
 - Zero initialization
 - Random dengan distribusi uniform.
 - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal)
 - Menerima parameter seed untuk reproducibility
 - Random dengan distribusi normal.
 - Menerima parameter mean dan variance
 - Menerima parameter seed untuk reproducibility
- Instance model yang diinisialisasikan harus bisa menyimpan bobot tiap neuron (termasuk bias)
- Instance model yang diinisialisasikan harus bisa menyimpan gradien bobot tiap neuron (termasuk bias)

- Instance model memiliki method untuk menampilkan model berupa struktur jaringan beserta bobot dan gradien bobot tiap neuron dalam bentuk graf. (Format graf dibebaskan)
- Instance model memiliki method untuk menampilkan distribusi bobot dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot
- Instance model memiliki method untuk menampilkan distribusi gradien bobot dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot
- Instance model memiliki method untuk *save* dan *load*
- Model memiliki implementasi forward propagation dengan ketentuan sebagai berikut:
 - Dapat menerima input berupa batch.
- Model memiliki implementasi backward propagation untuk menghitung perubahan gradien:
 - Dapat menangani perhitungan perubahan gradien untuk input data batch.
 - Gunakan konsep chain rule untuk menghitung gradien tiap bobot terhadap loss function.
 - Berikut merupakan turunan pertama untuk setiap fungsi aktivasi:

Nama Fungsi Aktivasi	Turunan Pertama
Linear	$\frac{d(\text{Linear}(x))}{dx} = 1$
ReLU	$\frac{d(\text{ReLU}(x))}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d(\sigma(x))}{dx} = \sigma(x)(1 - \sigma(x))$
Hyperbolic Tangent (tanh)	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x - e^{-x}} \right)^2$

Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $\frac{d(\text{softmax}(\vec{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(\text{softmax}(\vec{x})_1)}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(\text{softmax}(\vec{x})_n)}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x})_n)}{\partial x_n} \end{bmatrix}$ Dimana untuk $i, j \in \{1, \dots, n\}$, $\frac{\partial(\text{softmax}(\vec{x})_i)}{\partial x_j} = \text{softmax}(\vec{x})_i (\delta_{i,j} - \text{softmax}(\vec{x})_j)$ $\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$
---------	---

- Berikut merupakan turunan pertama untuk setiap fungsi loss terhadap bobot suatu FFNN (lanjutkan sisanya menggunakan chain rule):

Nama Fungsi Loss	Definisi Fungsi
MSE	$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \frac{\partial \hat{y}_i}{\partial W}$
Binary Cross-Entropy	$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \frac{\partial \hat{y}_i}{\partial W}$
Categorical Cross-Entropy	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{y_{ij}}{\hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W}$

- Model memiliki implementasi weight update dengan menggunakan gradient descent untuk memperbarui bobot berdasarkan gradien yang telah dihitung, berikut persamaannya:

$$W_{new} = W_{old} - \alpha \left(\frac{\partial \mathcal{L}}{\partial W_{old}} \right)$$

α = Learning rate

Pembahasan

2.1 Penjelasan Implementasi

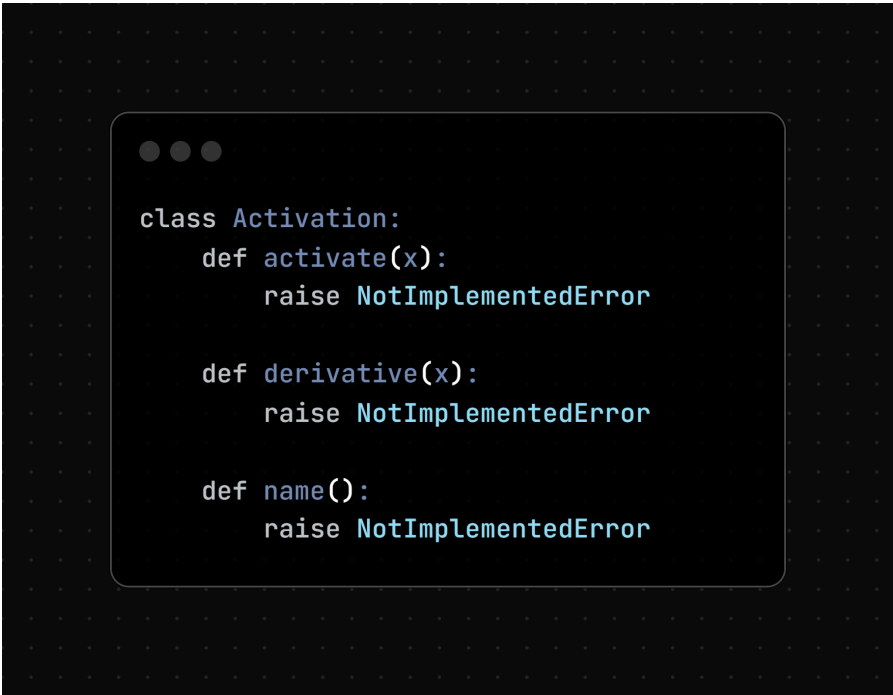
2.1.1 Deskripsi Kelas

2.1.1.1 Class Activation

Class Activation adalah kelas yang mengimplementasikan fungsi - fungsi aktivasi. Kelas ini memiliki 3 method, yaitu activate(x), derivative(x) dan name(), dengan x adalah nilai dari cross product.

Method activate adalah method yang mengembalikan nilai hasil dari aktivasi x. Method derivative adalah method yang mengembalikan nilai turunan dari fungsi aktivasi terhadap x. Method name adalah method yang mengembalikan nama dari class activation yang digunakan.

Class Activation memiliki banyak kelas turunan, yang masing - masing merepresentasikan fungsi aktivasi yang berbeda - beda. Berikut adalah implementasi Class Activation beserta contoh kelas turunannya.



```
class Activation:
    def activate(x):
        raise NotImplementedError

    def derivative(x):
        raise NotImplementedError

    def name():
        raise NotImplementedError
```

Class Activation


```

class Linear(Activation):
    def activate(x):
        return x

    def derivative(x):
        return np.ones_like(x)

    def name():
        return "Linear"

class ReLU(Activation):
    def activate(x):
        return np.maximum(0, x)

    def derivative(x):
        return np.where(x > 0, 1, 0)

    def name():
        return "ReLU"

```

Class Turunan dari Class Activation (Linear dan ReLU)

Class activation memiliki 5 class turunan yaitu, Linear, ReLU, Sigmoid, Tanh, dan Softmax.

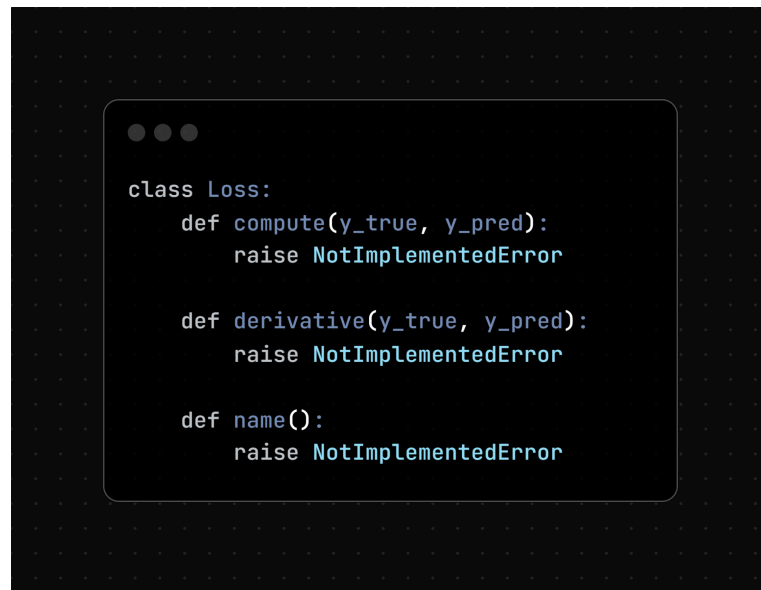
2.1.1.2 Class Losses

Class Loss adalah kelas yang mengimplementasikan fungsi - fungsi loss. Kelas ini memiliki 3 method, yaitu `compute(y_true, y_pred)`, `derivative(y_true, y_pred)` dan `name()`, dengan `y_true` dan `y_pred` untuk tiap class memiliki format yang berbeda - beda.

Nilai `y_true` dan `y_pred` pada fungsi loss MSE akan memiliki format numpy array dengan shape yang sama. Nilai `y_true` dan `y_pred` pada fungsi loss BinaryCrossEntropy akan memiliki format numpy array dengan binary values. Nilai `y_true` dan `y_pred` pada fungsi

loss CategoricalCrossEntropy akan memiliki format numpy array, dapat berupa 1D atau 2D dan y_pred adalah matrix probability distribution.

Class Losses memiliki banyak kelas turunan, yang masing - masing merepresentasikan fungsi loss yang berbeda - beda. Berikut adalah implementasi Class Losses beserta contoh kelas turunannya.

A screenshot of a code editor window with a dark background and a light gray grid pattern. The editor contains a Python class definition for 'Loss'. The class has three methods: 'compute', 'derivative', and 'name', each of which raises a 'NotImplementedError'. The code is written in a light blue font.

```
class Loss:
    def compute(y_true, y_pred):
        raise NotImplementedError

    def derivative(y_true, y_pred):
        raise NotImplementedError

    def name():
        raise NotImplementedError
```

Class Losses

```

class MSE(Loss):
    def compute(y_true, y_pred):
        return np.mean(np.square(y_true - y_pred))

    def derivative(y_true, y_pred):
        # Gradient descent, not negative (katanya)
        n = y_true.shape[0]
        return 2 * (y_pred - y_true) / n

    def name():
        return "MSE"

class BinaryCrossEntropy(Loss):
    def compute(y_real, y_pred):
        # Clip prevent log(0) (katanya)
        eps = 1e-15
        y_pred = np.clip(y_pred, eps, 1 - eps)
        return -np.mean(y_real * np.log(y_pred) + (1 - y_real) * np.log(1 - y_pred))

    def derivative(y_true, y_pred):
        # Clip prevent dbagi 0
        eps = 1e-15
        y_pred = np.clip(y_pred, eps, 1 - eps)

        n = y_true.shape[0]
        return (-y_true / y_pred + (1 - y_true) / (1 - y_pred)) / n

    def name():
        return "Binary Cross-Entropy"

class CategoricalCrossEntropy(Loss):
    def compute(y_true, y_pred):
        # Clip prevent log(0) (katanya)
        eps = 1e-15
        y_pred = np.clip(y_pred, eps, 1.0)

        # Binary
        if len(y_true.shape) == 2:
            return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))
        # Classes
        else:
            n = y_true.shape[0]
            return -np.sum(np.log(y_pred[np.arange(n), y_true])) / n

    def derivative(y_true, y_pred):
        eps = 1e-15
        y_pred = np.clip(y_pred, eps, 1.0)

        # Convert class ke binary if needed
        if len(y_true.shape) == 1:
            n_samples = y_true.shape[0]
            n_classes = y_pred.shape[1]
            y_true_one_hot = np.zeros((n_samples, n_classes))
            y_true_one_hot[np.arange(n_samples), y_true] = 1
            y_true = y_true_one_hot

        n = y_true.shape[0]
        return -y_true / (n * y_pred)

    def name():
        return "Categorical Cross-Entropy"

```

Class Turunan dari Class Losses

2.1.1.3 Class Initializers

Class Initializers adalah kelas yang mengimplementasikan fungsi - fungsi Inisialisasi weight. Kelas ini memiliki 2 method, yaitu `initialize(self, shape)`, dan `name()`, dengan shape adalah ukuran dari Neural Network yang weightnya akan diinisialisasi.

Class Initializers memiliki banyak kelas turunan, yang masing - masing merepresentasikan fungsi initializers yang berbeda - beda. Berikut adalah implementasi Class Initializers beserta contoh kelas turunannya.

```
class Initializer:
    def initialize(self, shape):
        raise NotImplementedError

    def name(self):
        raise NotImplementedError
```

Class Initializers

```
class ZeroInitializer(Initializer):
    def initialize(self, shape):
        return np.zeros(shape)

    def name(self):
        return "Zero Initializer"
```

Class Turunan dari Class Initializers

2.1.1.4 Class Regularizer

Class Regularizer adalah kelas abstrak yang mendefinisikan struktur dasar regularisasi dalam machine learning. Class ini memiliki 3 method, `compute(weights)`, `derivative(weights)` dan `name()` yang mengembalikan namanya. Kelas ini memiliki 2 turunan kelas

yaitu L1 Regularizer dan L2 Regularizer. Regularizer ini digunakan untuk mencegah overfitting

Berikut adalah implementasi dari kelas kelas tersebut.

```
1 class Regularizer:
2     def __init__(self, lambda_param=0.01):
3         self.lambda_param = lambda_param
4
5     def compute(self, weights):
6         raise NotImplementedError
7
8     def derivative(self, weights):
9         raise NotImplementedError
10
11    def name(self):
12        raise NotImplementedError
13
14
15    class L1Regularizer(Regularizer):
16        def compute(self, weights):
17            return self.lambda_param * np.sum([np.sum(np.abs(w)) for w in weights])
18
19        def derivative(self, weights):
20            return [self.lambda_param * np.sign(w) for w in weights]
21
22        def name(self):
23            return f"L1 Regularizer (lambda={self.lambda_param})"
24
25
26    class L2Regularizer(Regularizer):
27        def compute(self, weights):
28            return 0.5 * self.lambda_param * np.sum([np.sum(np.square(w)) for w in weights])
29
30        def derivative(self, weights):
31            return [self.lambda_param * w for w in weights]
32
33        def name(self):
34            return f"L2 Regularizer (lambda={self.lambda_param})"
```

Class Regularizer dan turunannya.

2.1.1.5 Class RMSNorm

Class RMSNorm adalah kelas untuk mengimplementasikan teknik normalisasi yang tidak bergantung pada batch. Atribut dari kelas ini adalah eps yaitu nilai kecil untuk menghindari pembagian 0, gamma sebagai parameter skala, grad_gamma sebagai gradien gamma yang didapatkan dalam backpropagation. Method yang terdapat dalam kelas ini adalah initialize(shape), forward(x), backward(grad_output) dan update(learning_rate).

Berikut adalah implementasi dari class RMSNorm

```
1 class RMSNorm:
2     def __init__(self, eps=1e-8):
3         self.eps = eps
4         self.gamma = None
5         self.grad_gamma = None
6
7     def initialize(self, shape):
8         self.gamma = np.ones(shape)
9
10    def forward(self, x):
11        self.x = x
12
13        self.rms = np.sqrt(np.mean(np.square(x), axis=1, keepdims=True) + self.eps)
14
15        self.normalized = x / self.rms
16
17        return self.gamma * self.normalized
18
19    def backward(self, grad_output):
20        self.grad_gamma = np.sum(grad_output * self.normalized, axis=0)
21
22        n_features = self.x.shape[1]
23        grad_input = grad_output * self.gamma / self.rms
24
25        grad_rms = -np.sum(grad_output * self.gamma * self.x / (self.rms ** 2), axis=1, keepdims=True)
26        grad_input += grad_rms * self.x / (n_features * self.rms)
27
28        return grad_input
29
30    def update(self, learning_rate):
31        self.gamma -= learning_rate * self.grad_gamma
```

Class RMSNorm

2.1.1.6 Class FFNN

Class FFNN adalah class model yang menjadi tempat pengimplementasian fungsi-fungsi inti dari Feedforward Neural Network. Kelas ini dirancang untuk menangani proses pembelajaran dengan mengimplementasikan fungsi-fungsi utama dalam Neural Network, seperti **forward propagation** untuk perhitungan keluaran berdasarkan bobot dan bias, serta **backward propagation** untuk menyesuaikan bobot berdasarkan kesalahan prediksi menggunakan algoritma optimasi. Class ini juga diimplementasikan seperti pada class model umumnya yang memiliki fungsi fit dan predict. Penjelasan atribut dari class ini adalah sebagai berikut.

Nama Attribute	Tipe Data	Keterangan
n_layer	integer	Banyak layer FFNN (termasuk input dan output layer)
layer_sizes	List[integer]	Jumlah neuron per layer dari input hingga output layer
activations	List[Activation]	List object Activation untuk tiap layer
loss	Loss	Object class Loss yang menyimpan instance Loss yang dipakai
regularizer	Regularizer	Object class Regularizer yang menyimpan instance Regularizer yang dipakai (L1 atau L2)
use_rms_norm	boolean	True jika memakai RMSNorm, false jika tidak memakai
Initializer	Initializer	Object class Initializer yang menyimpan instance Initializer yang dipakai
weights	List[ndarray2D (n_input, n_neuron) : Float]	Bentuk list yang menyimpan matriks weight tiap neuron dari tiap layer (kecuali input layer). Penyimpanan dalam bentuk matriks ini digunakan untuk perkalian matrix pada forward propagation. Contoh: Jika input layer memiliki banyak fitur 5 dan hidden layer pertama memiliki 7 neuron), maka bentuk weights dari hidden layer pertama adalah (5, 7), dengan nilai weight tiap neuron menempati satu kolom menurun ke bawah.
Biases	Biases : List[ndarray1D (n_neuron) : Float]	Bentuk list yang menyimpan nilai bias dari neuron-neuron pada tiap layer. Penyimpanan bias neuron-neuron pada satu layer disimpan dalam bentuk numpy array 1D yang berukuran sebanyak jumlah neuron pada layer tersebut. Contoh: Jika hidden layer pertama memiliki 5 neuron, maka biases layer tersebut merupakan ndarray 1D berukuran 5
weight_gradients	List[ndarray2D (n_input, n_neuron) : Float]	weight_gradients menyimpan perubahan bobot (weight) berdasarkan turunan fungsi loss terhadap weight $\frac{\partial L}{\partial W}$, untuk setiap weight pada setiap neuron di tiap layer. List ini berisi nilai weight gradient untuk setiap layer, di mana setiap elemennya merupakan array NumPy berdimensi 2

		yang menyimpan weight gradient untuk setiap neuron dalam layer tersebut.. Weight gradient ini merepresentasikan turunan Loss terhadap weight yang dilambangkan dengan simbol $\frac{\partial L}{\partial W}$.
Bias_gradients	List [ndarray1D (n_neuron) : Float]	Bias gradients menyimpan turunan fungsi loss terhadap bias di setiap lapisan. List menyimpan vector / array 1D yang berisi nilai bias untuk tiap neuron, pada tiap layer.
layer_inputs	ndarray2D (n_samples, n_features) : Float	Menyimpan nilai input layer yang berbentuk batch, dengan row menyatakan jumlah sampel pada batch dan kolom menyatakan jumlah feature input.
pre_activations	List[ndarray2D (n_inputs, n_features): Float]	Menyimpan nilai net yang diproses pada satu layer (sebelum dilewati fungsi aktivasi). Elemen dimulai dengan nilai net dari hidden layer pertama (exclude input layer)
Post_activations	List[ndarray2D (n_inputs, n_features) : Float]	Menyimpan nilai hasil aktivasi pada satu layer (output). Elemen dimulai dengan nilai output pada input layer (include input layer)

Untuk method-method dari Class FFNN, akan dijelaskan pada bagian berikut (Untuk penjelasan parameter input dapat dilihat pada komen di kode kami).

1. *Constructor*

```

1 def __init__(self, layer_sizes, activations, loss='mse',
2               weight_initializer='uniform', weight_init_params=None,
3               regularizer=None, use_rms_norm=False):

```

Method ini merupakan constructor dari FFNN dimana menginisiasi neural network dengan menerima input-input yang menjadi atribut dari model. Pertama, dilakukan validasi jumlah layer untuk memastikan bahwa jaringan minimal memiliki **dua layer** (input dan output). Selain itu, validasi jumlah fungsi aktivasi dilakukan karena input layer tidak memerlukan aktivasi.


```

1     if len(layer_sizes) < 2:
2         raise ValueError("Network must have at least 2 layers (input and output)")
3
4     if len(activations) != len(layer_sizes) - 1:
5         raise ValueError("Number of activations must be one less than number of layers")

```

Kemudian dilakukan inisialisasi model FFNN, yang menetapkan berbagai atribut yang menentukan struktur dan konfigurasi jaringan. Atribut-atribut ini mencakup ukuran dan jumlah layer, fungsi aktivasi, serta metode perhitungan loss yang akan digunakan. Selain itu, jika diperlukan, model juga menginisialisasi metode regularisasi untuk mencegah overfitting dan opsi **RMS Normalization** guna meningkatkan stabilitas pembelajaran.

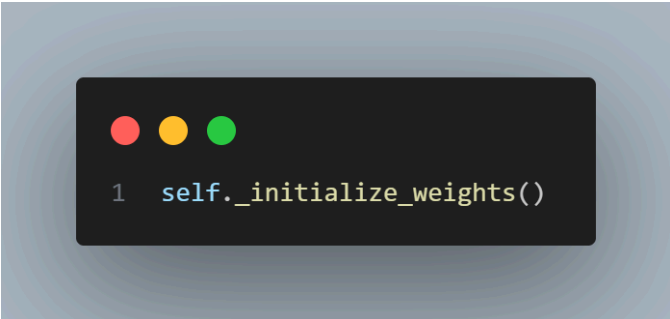
```

1  self.layer_sizes = layer_sizes
2  self.n_layers = len(layer_sizes)
3
4  self.activations = []
5  for act in activations:
6      if isinstance(act, str):
7          self.activations.append(get_activation(act))
8      else:
9          self.activations.append(act)
10
11 if isinstance(loss, str):
12     self.loss = get_loss(loss)
13 else:
14     self.loss = loss
15
16 self.regularizer = None
17 if regularizer:
18     if isinstance(regularizer, str):
19         self.regularizer = get_regularizer(regularizer,
20                                           **(weight_init_params or {}))
21     else:
22         self.regularizer = regularizer
23
24 self.use_rms_norm = use_rms_norm
25 self.normalizers = []
26 if use_rms_norm:
27     for i in range(self.n_layers - 1):
28         self.normalizers.append(RMSNorm())
29
30 if weight_init_params is None:
31     weight_init_params = {}
32
33 if isinstance(weight_initializer, str):
34     self.initializer = get_initializer(weight_initializer, **weight_init_params)
35 else:
36     self.initializer = weight_initializer

```

Untuk memastikan bobot jaringan diinisialisasi dengan baik, model menggunakan metode inisialisasi bobot yang dapat dikonfigurasi sesuai kebutuhan. Semua atribut ini diatur agar jaringan dapat berfungsi dengan optimal selama proses pelatihan dan inferensi.

Ctt: Method `initialize_weight` akan dijelaskan berikutnya



```
1 self._initialize_weights()
```

2. Method `initialize_weights`



```
1 def _initialize_weights(self):
2     self.weights = []
3     self.biases = []
4     self.weight_gradients = []
5     self.bias_gradients = []
6
7     for i in range(self.n_layers - 1):
8         weight_shape = (self.layer_sizes[i], self.layer_sizes[i+1])
9         self.weights.append(self.initializer.initialize(weight_shape))
10        self.weight_gradients.append(np.zeros(weight_shape))
11
12        bias_shape = (self.layer_sizes[i+1],)
13        self.biases.append(np.zeros(bias_shape))
14        self.bias_gradients.append(np.zeros(bias_shape))
15
16        # if rms norm
17        if self.use_rms_norm:
18            self.normalizers[i].initialize(bias_shape)
```

Metode `initialize_weights` bertanggung jawab untuk menginisialisasi bobot (`weights`) dan bias (`biases`) pada jaringan FFNN. Pertama, list kosong dibuat untuk menyimpan bobot, bias, serta gradiennya, yaitu `self.weights`, `self.biases`, `self.weight_gradients`, dan `self.bias_gradients`. Kemudian, melalui perulangan pada setiap layer kecuali output, bobot diinisialisasi dengan ukuran (jumlah neuron layer sebelumnya, jumlah neuron layer saat ini), menggunakan metode `self.initializer.initialize(weight_shape)` sesuai jenis normalizer yang diinisialisasi, lalu ditambahkan ke dalam `self.weights`.

Gradien bobot diatur sebagai array nol dengan bentuk yang sama dan disimpan dalam `self.weight_gradients`. Untuk bias, array nol dibuat dengan bentuk (jumlah neuron layer saat ini,) dan disimpan dalam `self.biases`, sedangkan `self.bias_gradients` juga diinisialisasi dengan array

nol. Jika opsi RMS Normalization diaktifkan, setiap normalizer pada layer yang bersangkutan juga diinisialisasi dengan bentuk yang sesuai menggunakan `self.normalizers[i].initialize(bias_shape)`.

3. Method *Forward* (Akan dijelaskan pada bagian *Forward Propagation*)
4. Method *Backward* (Akan dijelaskan pada bagian *Backward Propagation*)
5. Method *update_weights* (Akan dijelaskan pada bagian *Update Weight*)
6. Method *Fit*

```
1 def fit(self, X, y, batch_size=32, learning_rate=0.01, epochs=100,  
2     verbose=1, validation_data=None):
```

Method `fit` pada class FFNN digunakan untuk **melatih jaringan saraf** dengan **menerapkan forward propagation, backpropagation, dan pembaruan bobot** menggunakan gradient descent. Berikut adalah penjelasan rinci mengenai prosesnya.

```
1 n_samples = X.shape[0]  
2 history = {  
3     'train_loss': [],  
4     'val_loss' : [] if validation_data is not None else None  
5 }
```

Pertama kita menginisialisasi variabel yang akan menyimpan history dari train loss dan validation loss (jika ada). Variabel `history` ini digunakan untuk mengetahui status training network pada tiap epoch. Kemudian kita lakukan training dengan melakukan iterasi epoch dimana dalam satu epoch, kita akan melakukan pelatihan neural network dengan forward propagation, backpropagation, dan update weight.

```

1  for epoch in range(epochs):
2      start_time = time.time()
3      epoch_loss = 0.0
4
5      iterator = batch_iterator(X,y, batch_size)
6      n_batches = int(np.ceil(n_samples/batch_size))
7
8      for batch_idx, (X_batch, y_batch) in enumerate(iterator):
9          if verbose == 1 and batch_idx % max(1,n_batches // 10) == 0:
10             print(f"Epoch {epoch+1}/{epochs} - Batch {batch_idx+1}/{n_batches}")
11
12             self.forward(X_batch)
13
14             batch_loss = self.backward(y_batch)
15             epoch_loss += batch_loss * X_batch.shape[0] / n_samples
16
17             self.update_weights(learning_rate)
18
19     history['train_loss'].append(epoch_loss)

```

Pada tiap epoch, data X train kita bagi menjadi beberapa batch kecil. Kita inisialisasi epoch loss dengan nilai 0. Kemudian pada tiap batch kecil yang dihasilkan, kita lakukan forward, backward, dan update weight. Nilai epoch loss ditambah dengan nilai batch loss dikali dengan proporsi jumlah row dibanding jumlah X train keseluruhan. Perkalian proporsi ini bertujuan agar tiap batch memberikan kontribusi pada epoch loss yang sebanding dengan banyak row dalam batch tersebut. Setelah semua batch selesai, nilai epoch loss tadi kita tambahkan pada history train loss.

```

1      if validation_data is not None:
2          X_val, y_val = validation_data
3          y_val = ensure_2d_y(y_val, self.activations[-1].name())
4
5          y_pred = self.forward(X_val)
6          val_loss = self.loss.compute(y_val, y_pred)
7          history['val_loss'].append(val_loss)

```

Kemudian kita lakukan forward propagation dengan validation data dengan tujuan hanya untuk melihat loss value per epoch nya. Untuk validation data, tidak ada dilakukan proses pembelajaran dengan backward/update weight. Kemudian setelah melakukan forward / predict, kita catat nilai validation data loss nya pada variabel history.

Proses ini dilakukan berulang hingga mencapai max epoch. Setelah iterasi mencapai max epoch, proses fitting / training selesai.

7. Method *predict*

Method ini merupakan interface saat ingin melakukan prediksi dengan FFNN yang telah ditrain. Method ini hanya sebagai perantara yang memanggil method *forward* dengan X train dari input.

```
1 def predict(self, X):
2     """
3     Make predictions
4
5     Parameters:
6     -----
7     X : ndarray of shape (n_samples, n_features)
8         Input data
9
10    Returns:
11    -----
12    ndarray
13        Predictions
14    """
15    return self.forward(X)
```

8. Method *evaluate*

Method ini digunakan untuk menghitung loss value dari hasil prediksi model FFNN sekarang dengan input berupa X train dan y target. Method ini digunakan untuk mengevaluasi performa dari model FFNN.

```
1 def evaluate(self, X, y):
2     """
3     Evaluate the model
4
5     Parameters:
6     -----
7     X : ndarray of shape (n_samples, n_features)
8         Input data
9
10    y : ndarray of shape (n_samples, n_outputs) or (n_samples,)
11        Target values
12
13    Returns:
14    -----
15    float
16        Loss value
17    """
18    y = ensure_2d_y(y, self.activations[-1].name())
19
20    y_pred = self.predict(X)
21    val_loss = self.loss.compute(y, y_pred)
22    return val_loss
```

9. Method *save*

Method ini berguna untuk menyimpan model dan state yang telah diserialisasikan menjadi file menggunakan modul *pickle*

```
1 def save(self, file_path):
2     """
3     Save model to file
4
5     Parameters:
6     -----
7     file_path : str
8         Path to save file
9     """
10    with open(file_path, 'wb') as f:
11        pickle.dump({
12            'layer_sizes': self.layer_sizes,
13            'weights': self.weights,
14            'biases': self.biases,
15            'activations': self.activations,
16            'loss': self.loss,
17            'regularizer': self.regularizer,
18            'use_rms_norm': self.use_rms_norm
19        }, f)
```

10. Method *load*

Method ini digunakan untuk meload state dan model FFNN dari file hasil save dengan modul *pickle*

```
1 def load(cls, file_path):
2     """
3     Load model from file
4
5     Parameters:
6     -----
7     file_path : str
8         Path to model file
9
10    Returns:
11    -----
12    FFNN
13        Loaded model
14    """
15    with open(file_path, 'rb') as f:
16        model_data = pickle.load(f)
17
18    model = cls(
19        layer_sizes=model_data['layer_sizes'],
20        activations=model_data['activations'],
21        loss=model_data['loss']
22    )
23
24    model.weights = model_data['weights']
25    model.biases = model_data['biases']
26    model.regularizer = model_data.get('regularizer')
27    model.use_rms_norm = model_data.get('use_rms_norm', False)
28
29    return model
```

11. Method Fungsional Plotting

Terdapat beberapa method extra untuk plotting pada class FFNN, yakni

- Method `plot_model`

Memvisualisasikan arsitektur network dengan weight-weight dari tiap neuron

```
1 def plot_model(self):
2     """
3     Visualize the network architecture with weights
4     """
5     from .utils import plot_network_graph
6     plot_network_graph(self.layer_sizes, self.weights, self.biases,
7                        title='Neural Network Architecture')
```

- Method `plot_weight`

Memvisualisasikan distribusi weight pada layer tertentu

```
1 def plot_weight(self, layers=None):
2     """
3     Plot weight distribution of specified layers
4
5     Parameters:
6     -----
7     layers : list of int or None
8             Indices of layers to plot. If None, plot all layers.
9     """
10    if layers is None:
11        layers = list(range(len(self.weights)))
12
13    weights_to_plot = [self.weights[i] for i in layers]
14    layer_names = [f'Layer {i+1}' for i in layers]
15
16    from .utils import plot_weight_distribution
17    plot_weight_distribution(weights_to_plot,
```

- Method `plot_gradient_distribution`

Memvisualisasikan plot gradient distribution pada layer tertentu.

```

1  def plot_gradient_distribution(self, layers=None):
2      """
3      Plot gradient distribution of specified layers
4
5      Parameters:
6      -----
7      layers : list of int or None
8          Indices of layers to plot. If None, plot all layers.
9      """
10     if layers is None:
11         layers = list(range(len(self.weight_gradients)))
12
13     gradients_to_plot = [self.weight_gradients[i] for i in layers]
14     layer_names = [f'Layer {i+1}' for i in layers]
15
16     from .utils import plot_weight_distribution
17     plot_weight_distribution(gradients_to_plot,
18                             title='Gradient Distribution')

```

2.1.2 Forward Propagation

```

1  def forward(self, X):
2      self.layer_inputs = [X]
3      self.pre_activations = [] # Net
4      self.post_activations = [X] # Output
5
6      for i in range(self.n_layers-1):
7          net = np.dot(self.post_activations[-1], self.weights[i]) + self.biases[i]
8          self.pre_activations.append(net)
9
10         if self.use_rms_norm:
11             net = self.normalizers[i].forward(net)
12
13         output = self.activations[i].activate(net)
14         self.post_activations.append(output)
15
16     return self.post_activations[-1]

```

Metode *forward* menjalankan **forward propagation**, yaitu proses perhitungan keluaran jaringan berdasarkan input yang diberikan (X). Untuk setiap layer (kecuali layer input), dilakukan perhitungan

Net, yaitu kombinasi linier antara output layer sebelumnya (matriks input) dengan bobot dan bias pada layer saat ini.

$$net = X_i \cdot W_i + b_i$$

X berupa matriks input, W_i berupa matriks weight layer $- i$, b_i berupa vector bias layer $- i$

Proses ini dilakukan bertahap dari hidden layer awal hingga layer terakhir (output layer). Jika menggunakan RMS, maka nilai net akan dinormalisasi terlebih dahulu. Kemudian nilai net disimpan pada array karena akan digunakan pada proses backward propagation. Nilai net tadi kemudian dilewati fungsi aktivasi sehingga menjadi nilai output (post activation). Nilai output ini juga kemudian disimpan pada array.

Fungsi *forward* akan mereturn nilai prediksi dari output layer yang dimana berupa element terakhir `post_activations` (hasil output layer).

2.1.3 Backward Propagation

```
1 def backward(self, y_true):
2     y_pred = self.post_activations[-1]
3
4     loss_value = self.loss.compute(y_true, y_pred)
5
6     if self.regularizer:
7         reg_loss = self.regularizer.compute(self.weights)
8         loss_value += reg_loss
9
10    batch_size = y_true.shape[0]
11
12    is_special_case = (self.loss.name() == "Categorical Cross-Entropy" and
13                      self.activations[-1].name() == "Softmax") or (
14                      self.loss.name() == "Binary Cross-Entropy" and
15                      self.activations[-1].name() == "Sigmoid")
16
17    if is_special_case:
18        delta = y_pred - y_true
19    else:
20        loss_gradient = self.loss.derivative(y_true, y_pred)
21        loss_gradient = np.clip(loss_gradient, -1e10, 1e10)
22        delta = self._mult_activation_derivative(
23            loss_gradient, self.pre_activations[-1], self.activations[-1])
24
25    for i in range(self.n_layers - 2, -1, -1):
26        if self.use_rms_norm:
27            delta = self.normalizers[i].backward(delta)
28
29        delta_clipped = np.clip(delta, -1e10, 1e10)
30        self.weight_gradients[i] = np.dot(self.post_activations[i].T, delta_clipped)
31        self.bias_gradients[i] = np.sum(delta_clipped, axis=0)
32
33        if self.regularizer:
34            reg_grad = self.regularizer.derivative([self.weights[i]])[0]
35            self.weight_gradients[i] += reg_grad
36
37        if i > 0:
38            delta = np.dot(delta_clipped, self.weights[i].T)
39            delta = np.clip(delta, -1e10, 1e10)
40            delta = self._mult_activation_derivative(
41                delta, self.pre_activations[i-1], self.activations[i-1])
42
43    return loss_value
```

Method *backpropagation* adalah implementasi pada class FFNN untuk melakukan pembelajaran dan melatih neural network. Ini bekerja dengan cara menyebarkan error dari output ke arah input melalui jaringan, dan memperoleh nilai turunan dari fungsi loss terhadap weight (weight gradient) yang kemudian akan digunakan untuk memperbarui weight dari tiap layer.

Pertama, kita akan memulai pencarian weight gradien dari layer terakhir, yakni output layer. Kita akan mencari nilai delta (“*error term*”) terlebih dahulu pada output layer dengan rumus berikut.

$$\delta_o = \frac{\delta L}{\delta z_o} = \frac{\delta L}{\delta a_o} \cdot f'(z_o)$$

δ_o : delta / error term output layer

$\frac{\delta L}{\delta z}$: turunan fungsi Loss terhadap net (pre activation)

$\frac{\delta L}{\delta a}$: turunan fungsi Loss terhadap output (post activation)

$f'(z)$: turunan fungsi aktivasi

Akan tetapi, sebelumnya kita perlu memperhatikan bahwa terdapat beberapa variasi rumus *error term* output layer ini untuk *Loss Function* dan *Activation Function* yang berbeda. Berikut adalah penjelasannya.

1. **Softmax** Activation Function dan **Categorical Cross Entropy** Loss Function

Untuk kombinasi ini, nilai dari *delta* output layer dapat langsung kita hitung sebagai berikut. Hal ini karena proses perkalian kedua turunan fungsi ini menghasilkan fungsi akhir yang sederhana, sehingga mempermudah proses komputasi.

$$\delta_o = \hat{y} - y$$

dengan $\hat{y}$ adalah *predicted value*, dan y adalah *target true value*

2. **Sigmoid** Activation Function dan **Binary Cross Entropy** Loss Function

Kombinasi ini memiliki nilai error term yang sama seperti pada kombinasi *Softmax* dan *Categorical Cross Entropy*.

Untuk kombinasi loss function dan activation function selain yang disebutkan di atas, perkalian dengan turunan aktivasi mengikuti aturan sebagai berikut. (Aturan ini bukan hanya akan digunakan pada perhitungan *delta output*, melainkan juga *delta hidden*)

1. **Softmax** Activation, **other** Loss Function

Untuk fungsi aktivasi *Softmax* dengan Loss Function selain *Categorical Cross Entropy*, kita lakukan *sum* / perkalian matriks antara nilai loss function dengan *jacobian matrix* hasil turunan dari *Softmax*.

2. **Other Activation** function

Untuk kasus ini, perhitungan delta mengikuti rumus tersebut dan jenis perkalian adalah *element wise*

```

1  def _mult_activation_derivative(self, temp_delta, y_pred, activation):
2      if (activation.name() != "Softmax"):
3          return temp_delta * activation.derivative(y_pred)
4
5      jacobian = activation.derivative(y_pred)
6
7      new_delta = np.zeros_like(temp_delta)
8
9      for i in range(temp_delta.shape[0]):
10         new_delta[i] = np.dot(temp_delta[i], jacobian[i])
11
12     return new_delta

```

Kemudian akan dihitung *weight gradient* dari layer output yang diperoleh dengan rumus berikut (rumus ini juga berlaku untuk perhitungan weight gradient di layer hidden).

$$\frac{\partial L}{\partial W_l} = \delta_l \cdot (a_{l-1})^T \quad (i)$$

dengan $\frac{\partial L}{\partial W_l}$ merupakan weight gradient layer l ,

δ_l adalah delta layer l , dan

$(a_{l-1})^T$ adalah hasil output layer sebelumnya.

Setelah weight gradient, kita juga menghitung bias gradient dari layer tersebut dengan nilai berupa mean dari *delta*.

Kemudian kita lakukan perulangan untuk mencari nilai *delta*, weight gradien, dan bias gradien dari tiap layer hidden mulai dari hidden layer terakhir hingga hidden layer pertama. Perhitungan *delta* mengikuti rumus berikut,

$$\delta_l = (W_{l+1})^T \cdot \delta_{l+1} \cdot f'(z_l)$$

dengan $(W_{l+1})^T$ merupakan weight pada layer $l+1$

Setelah didapat *delta* dari hidden layer, kemudian kita cari weight gradien dan bias gradien untuk hidden layer tersebut dengan rumus weight gradient yang sama seperti pada output layer (lihat rumus (i))

Setelah proses backward selesai hingga hidden layer pertama, kita menghitung *loss* value dari hasil prediksi pada forward propagation dan y *true*. Perhitungan ini tidak berbeda ketika dilakukan sebelum maupun setelah proses backward karena pemrosesan menggunakan batch. *Loss* value ini yang kemudian akan direturn untuk digunakan pada proses fitting sebagai indikator progress pelatihan.

2.1.4 Update Weight

```
1 def update_weights(self, learning_rate, gradient_clip=1.0):
2     """
3     Update weights and biases using gradient descent with clipping
4
5     Parameters:
6     -----
7     learning_rate : float
8         Learning rate for gradient descent
9     gradient_clip : float
10        Maximum gradient norm
11    """
12    self.clip_gradients(max_norm=gradient_clip)
13
14    for i in range(self.n_layers - 1):
15        epsilon = 1e-8
16
17        self.weights[i] -= learning_rate * self.weight_gradients[i]
18
19        self.biases[i] -= learning_rate * self.bias_gradients[i]
20
21        if self.use_rms_norm:
22            self.normalizers[i].update(learning_rate)
```

Pada method ini, kami akan melakukan update weight dari tiap layer dengan menggunakan nilai *weight gradient* yang telah kami dapat pada proses *backward propagation*. Kita akan melakukan iterasi layer dari hidden layer pertama hingga layer terakhir (output layer). Proses update ini mengikuti rumus seperti berikut.

$$W_l = W_l - \eta \frac{\partial L}{\partial W_l}$$

dengan η adalah learning rate, $\frac{\partial L}{\partial W_l}$ adalah weight gradient untuk layer l .

Sedangkan untuk bias mengikuti rumus yang sama seperti pada perubahan weight, yakni sebagai berikut

$$b_l = b_l - \eta \frac{\partial L}{\partial W_l}$$

dengan η adalah learning rate, $\frac{\partial b}{\partial W_l}$ adalah bias gradient untuk layer l .

Selain dari kelas - kelas yang kami sebutkan diatas terdapat beberapa fungsi tambahan yang digunakan untuk mempermudah pembuatan model FFNN ini, beberapa diantaranya adalah fungsi - fungsi one hot encoding, batch iterator, dan fungsi plotting yang digunakan untuk melakukan plotting weight, loss, dan gambar dari neural network kami.

2.2 Pengujian

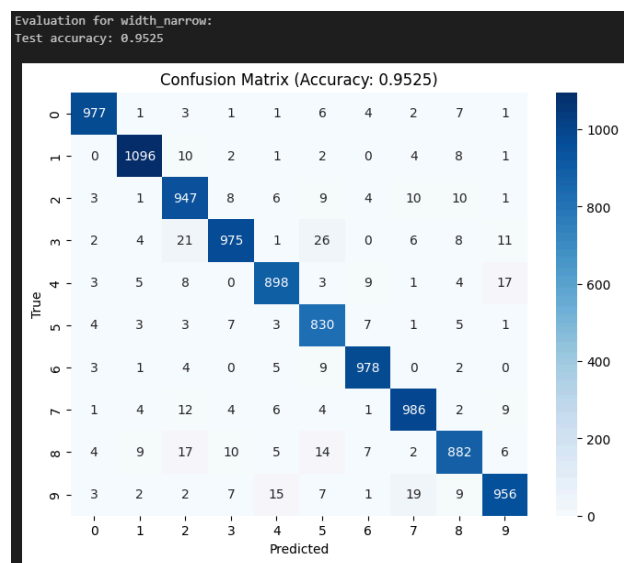
Seluruh Pengujian akan menggunakan dataset openml mnist_784 dengan jumlah sample yang digunakan sebanyak 50000 sample. 50000 sample ini akan dibagi menjadi train dan test sample yaitu sebanyak 40000 train sample dan 10000 test sample. Dikarenakan data pengujian menggunakan openml mnist_784, maka jumlah neuron pada input layer adalah 784 dan jumlah neuron pada output layer sebanyak 10. Keseluruhan pengujian akan menggunakan beberapa parameter yang sama, diantaranya adalah **learning rate** sebesar **0.01**, dan jumlah **epoch** sebanyak **20** epoch.

2.2.1 Pengaruh Depth dan Width

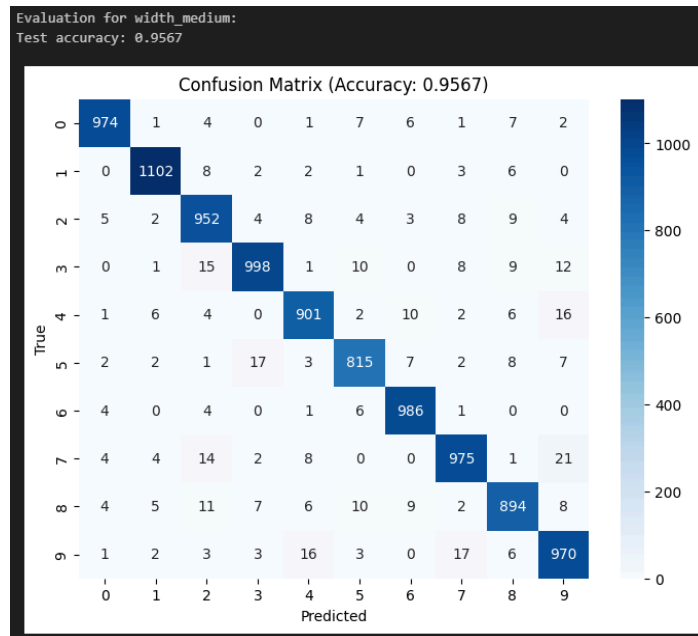
Pengujian pengaruh Depth dan Width pada model akan dilakukan dengan menggunakan 3 variasi untuk width dan 3 variasi untuk depth.

Variasi width yang digunakan adalah **64** neuron, **128** neuron, dan **256** neuron, dengan jumlah layer adalah 5, input layer, output layer, dan 3 hidden layer. Fungsi aktivasi yang digunakan pada hidden layer adalah ReLU dan softmax pada output layer, dan loss function yang digunakan adalah categorical cross entropy loss function. Weight Initializer yang digunakan adalah **xavier**.

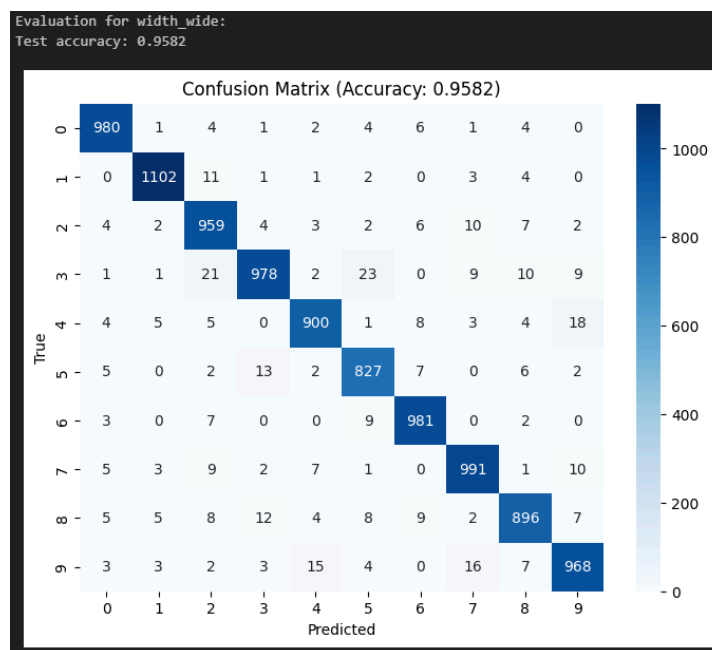
Berikut adalah hasil pengujian Width



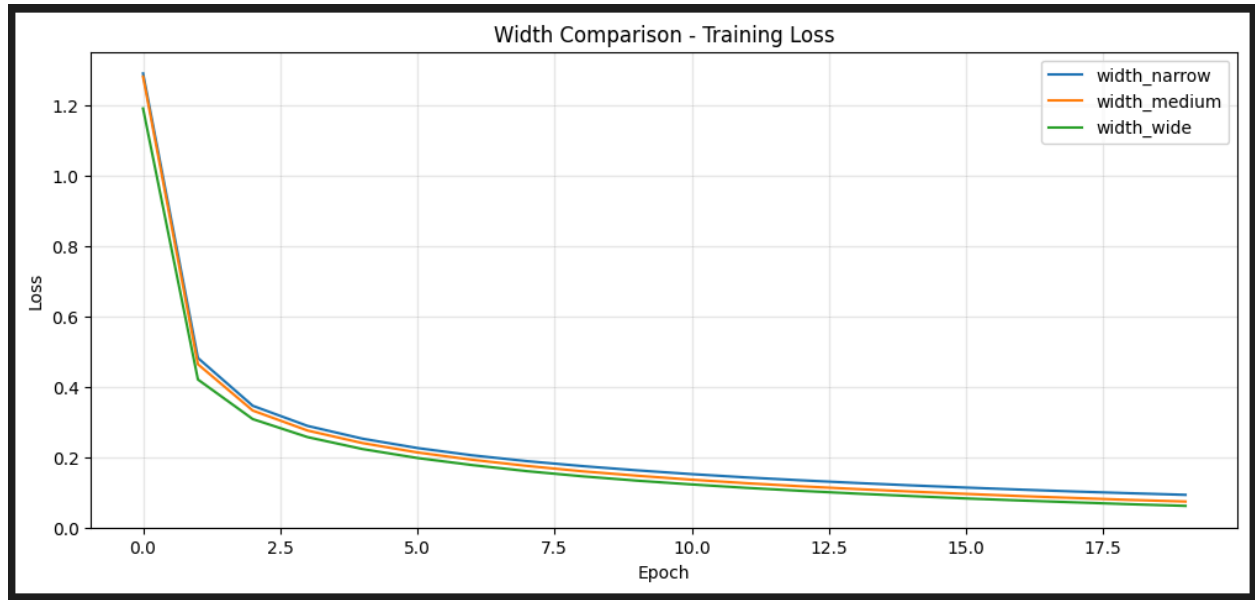
Width 64



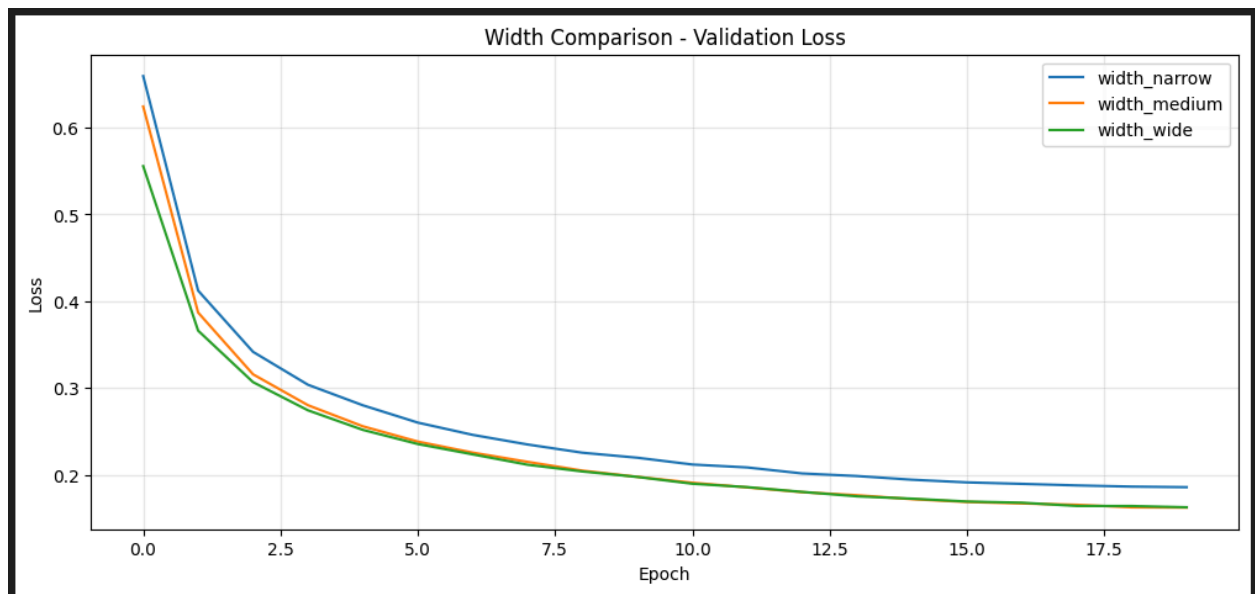
Width 128



Width 256



Training Loss Pengujian Width

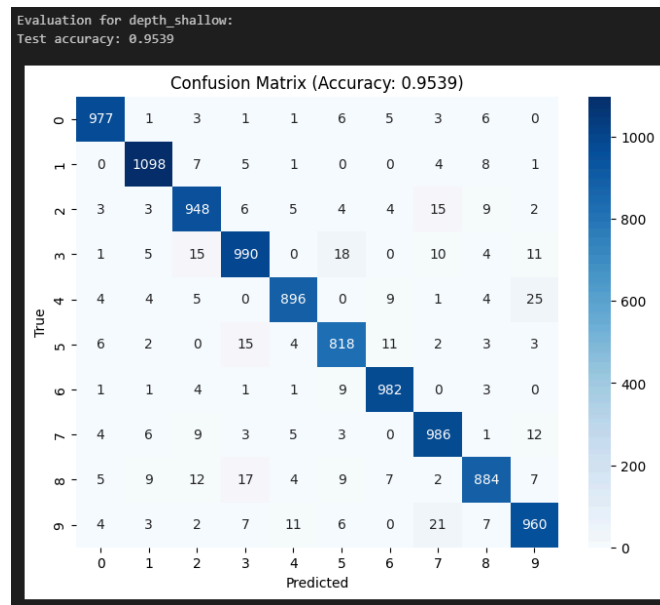


Validation Loss Pengujian Width

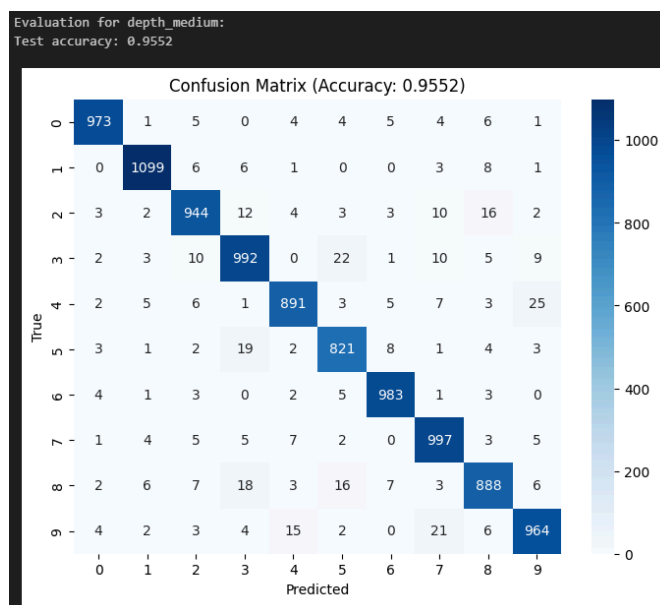
Berdasarkan hasil pengujian Width, dapat dilihat akurasi validation **meningkat** dengan semakin tingginya width dari hidden layer, dengan akurasi tertinggi didapatkan pada width 256 yaitu **0.9582**. Hal ini dikarenakan semakin banyaknya neuron dalam hidden layer, semakin banyak pola yang didapatkan, tiap neuron mempelajari representasi yang berbeda dari data sehingga dapat memungkinkan untuk mempelajari relasi relasi yang kompleks antar fitur, semakin kecil jumlah neuron, semakin memungkinkan untuk model melakukan **underfit** terhadap data.

Variasi depth yang digunakan adalah 1 hidden layer, 4 hidden layer, dan 8 hidden layer. Fungsi aktivasi yang digunakan pada hidden layer adalah ReLU dan softmax pada output layer, dan loss function yang digunakan adalah categorical cross entropy loss function. Weight Initializer yang digunakan adalah **he**.

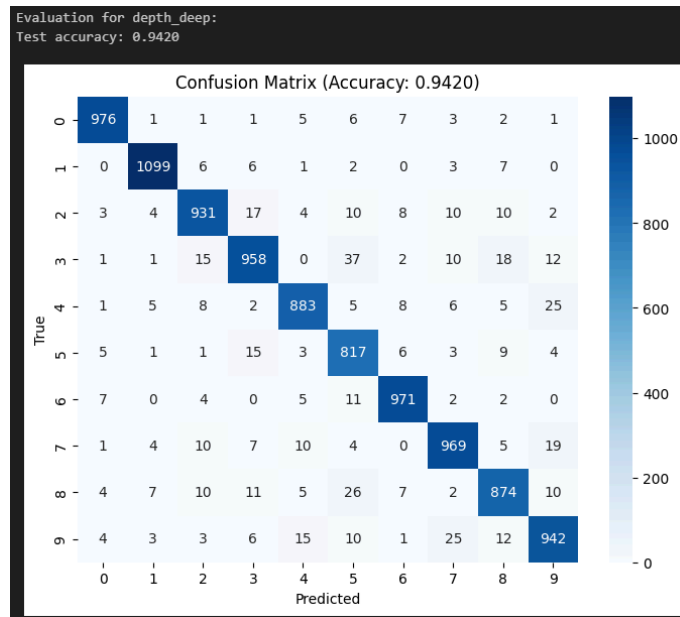
Berikut adalah hasil pengujian Depth



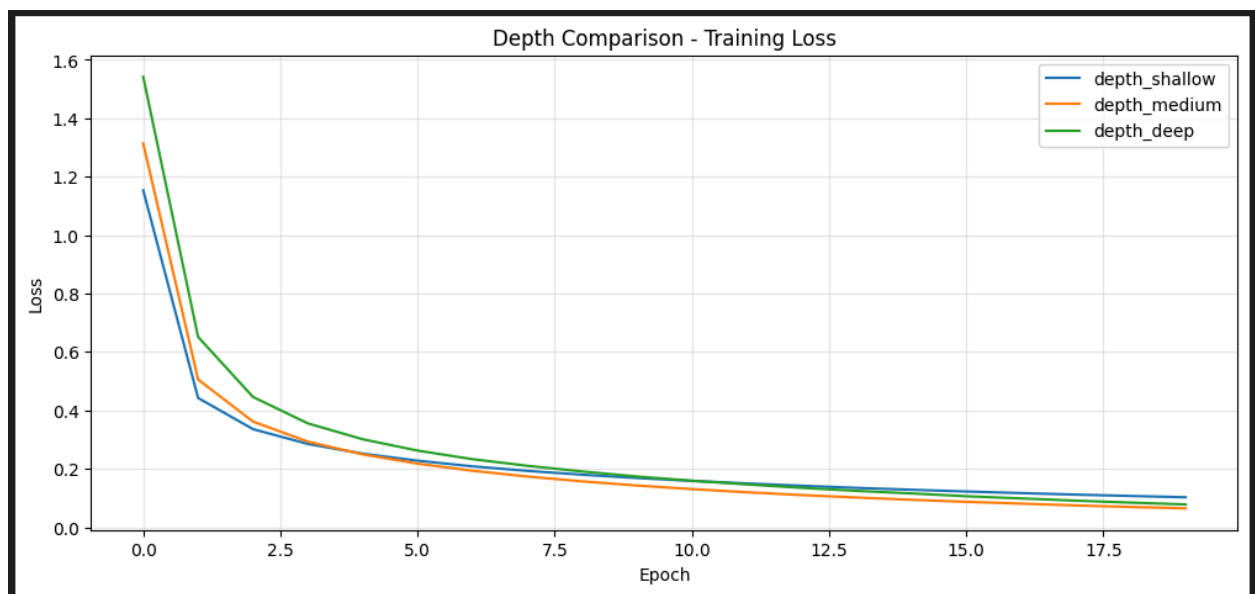
Depth 1



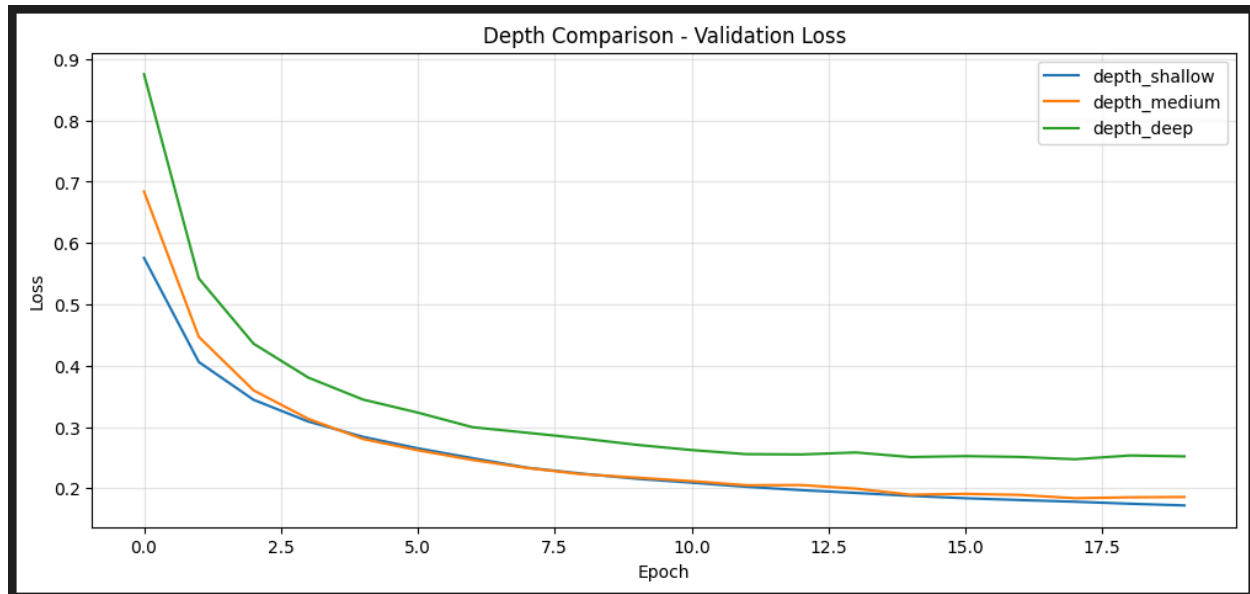
Depth 4



Depth 8



Training Loss Pengujian Depth



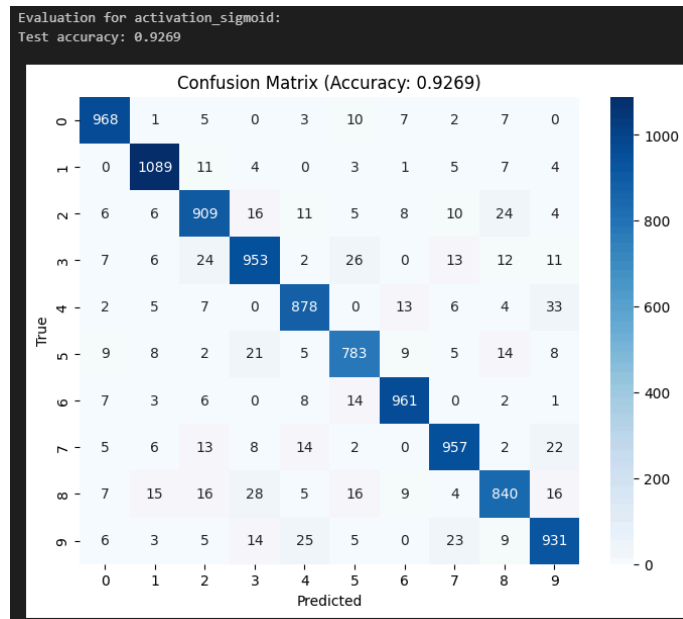
Validation Loss Pengujian Depth

Berdasarkan hasil pengujian Depth, dapat dilihat akurasi validation **meningkat** dengan semakin tingginya depth dari hidden layer, tetapi akurasi yang tertinggi diantara ketiga contoh ini adalah contoh dengan depth 4. Terdapat beberapa hal yang mempengaruhi mengapa akurais pada depth 8 lebih rendah dari depth 4. Dapat dilihat dari graf training dan validation loss bahwa loss depth 8 pada akhirnya mendekati loss dari depth 4, namun pada graf validation loss, graf depth 8 secara konsisten berada di atas kedua graf lain, dan juga cukup fluktuatif. Hal ini menandakan adanya overfitting, yang mana model tersebut cukup baik ketika digunakan di data training, namun tidak begitu baik ketika digunakan pada data validation. Hal ini dikarenakan depth yang lebih besar memberikan kompleksitas yang lebih besar juga, deep network pada depth 8 dapat menangkap noise dan bukan pola yang benar.

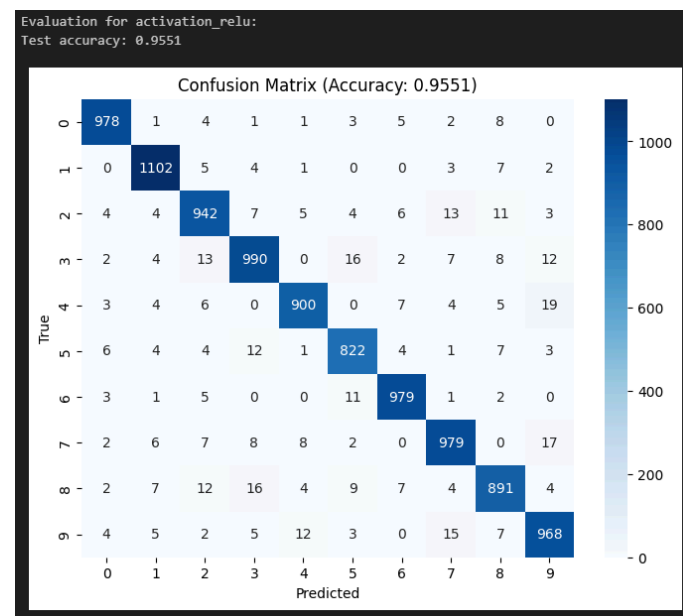
2.2.2 Pengaruh Fungsi Aktivasi

Pengujian pengaruh fungsi aktivasi pada model akan dilakukan dengan menggunakan **1** hidden layer dengan **128** neuron. Loss function yang digunakan adalah **categorical cross entropy** loss function dan khusus untuk fungsi aktivasi sigmoid akan menggunakan **binary cross entropy** loss function . Weight Initializer yang digunakan adalah **he**.

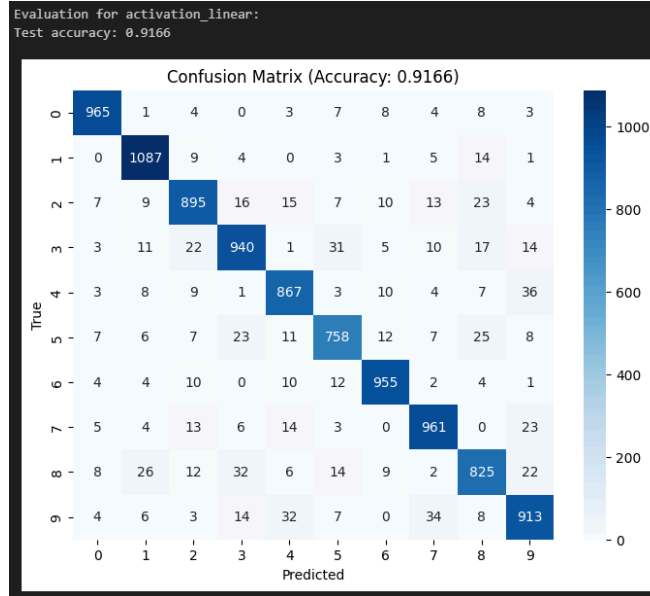
Berikut adalah hasil pengujian pengaruh fungsi aktivasi



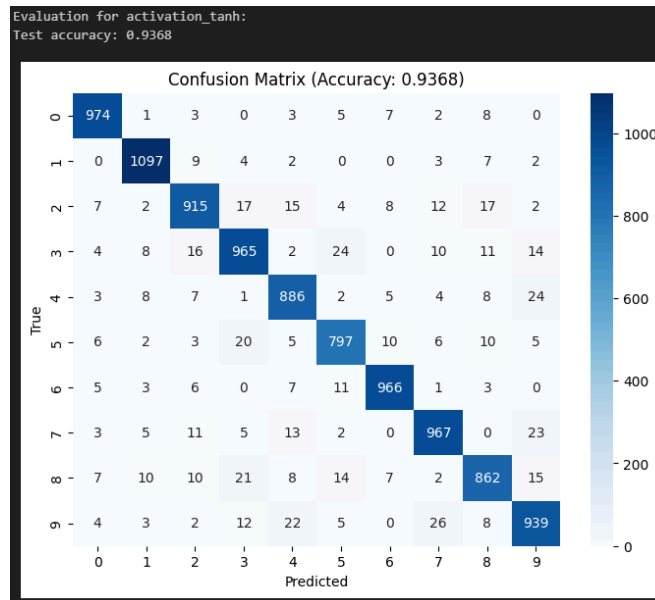
Fungsi Aktivasi Sigmoid



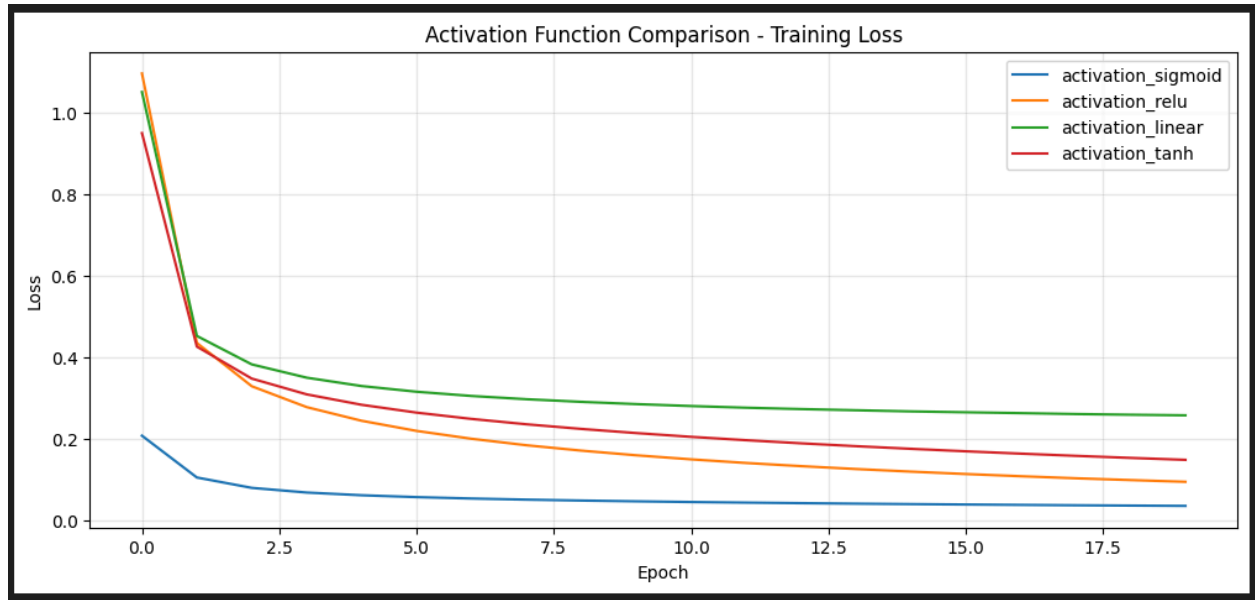
Fungsi Aktivasi ReLU



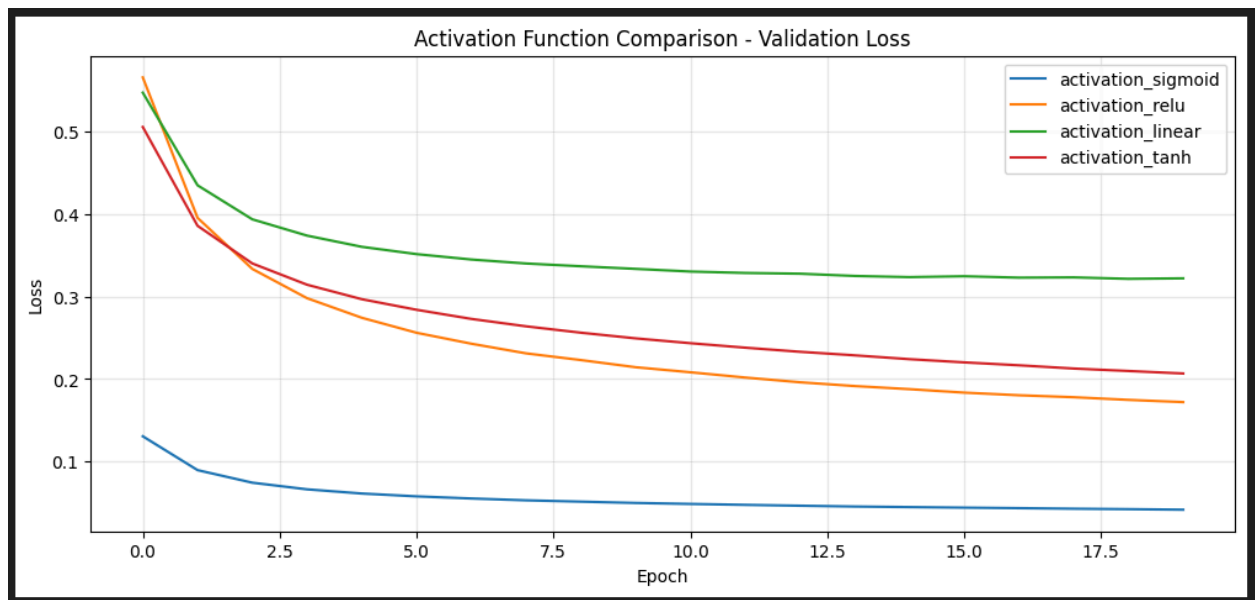
Fungsi Aktivasi Linear



Fungsi Aktivasi tanh



Training Loss Pengujian Fungsi Aktivasi



Validation Loss Pengujian Fungsi Aktivasi

Didapatkan hasil dari eksperimen adalah nilai akurasi sebagai berikut:

- Sigmoid: 0.9269
- ReLU: 0.9551
- Linear: 0.9166
- Tanh: 0.9368

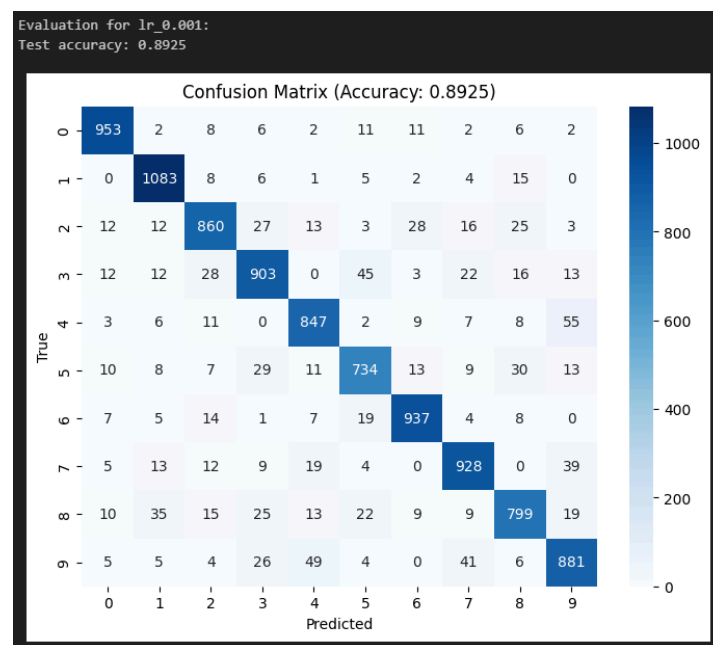
ReLU mendapatkan akurasi paling tinggi, hal ini dikarenakan ReLU menangani vanishing gradient problem, tidak seperti model model lainnya terutama tanh, mudah mengalami vanishing

gradient problem, terutama pada deep network dikarenakan nilai turunan *tanh* menjadi sangat kecil ketika terdapat input positif atau negatif yang besar. Fungsi sigmoid berada pada urutan ketiga, terbukti kurang efektif dikarenakan membatasi outputnya pada antara 0 dan 1 sehingga pada network yang kompleks menjadi kurang efektif. Selain itu, data yang diklasifikasikan pada tugas ini bukanlah data biner, sehingga membuat performa fungsi sigmoid menjadi lebih rendah lagi. Fungsi linear terdapat pada urutan terakhir dikarenakan pada dasarnya jika menggunakan fungsi ini, hanya akan mengubah model neural network menjadi beberapa regresi linear yang lebih sederhana.

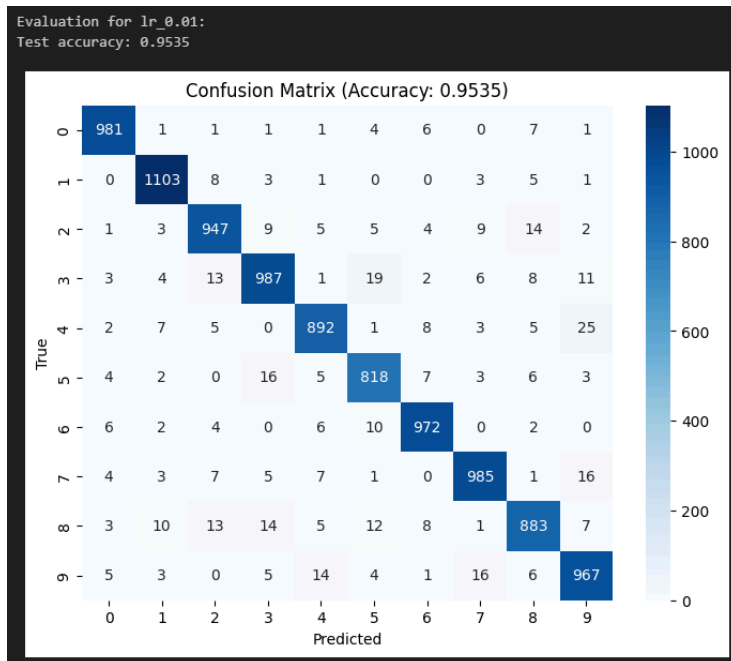
2.2.3 Pengaruh Learning Rate

Pengujian pengaruh learning rate pada model akan dilakukan dengan menggunakan **1** hidden layer dengan **128** neuron. Loss function yang digunakan adalah **categorical cross entropy** loss function . Weight Initializer yang digunakan adalah **he**.

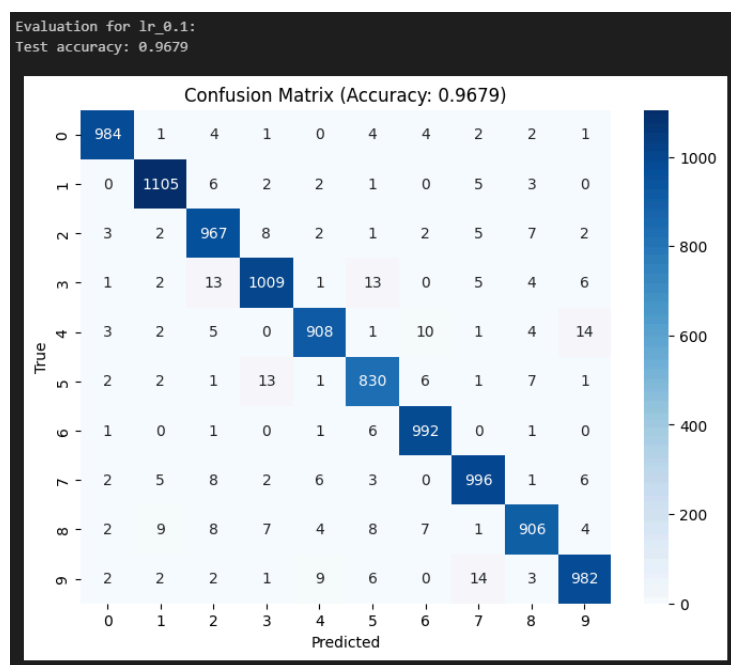
Berikut adalah hasil pengujian pengaruh learning rate



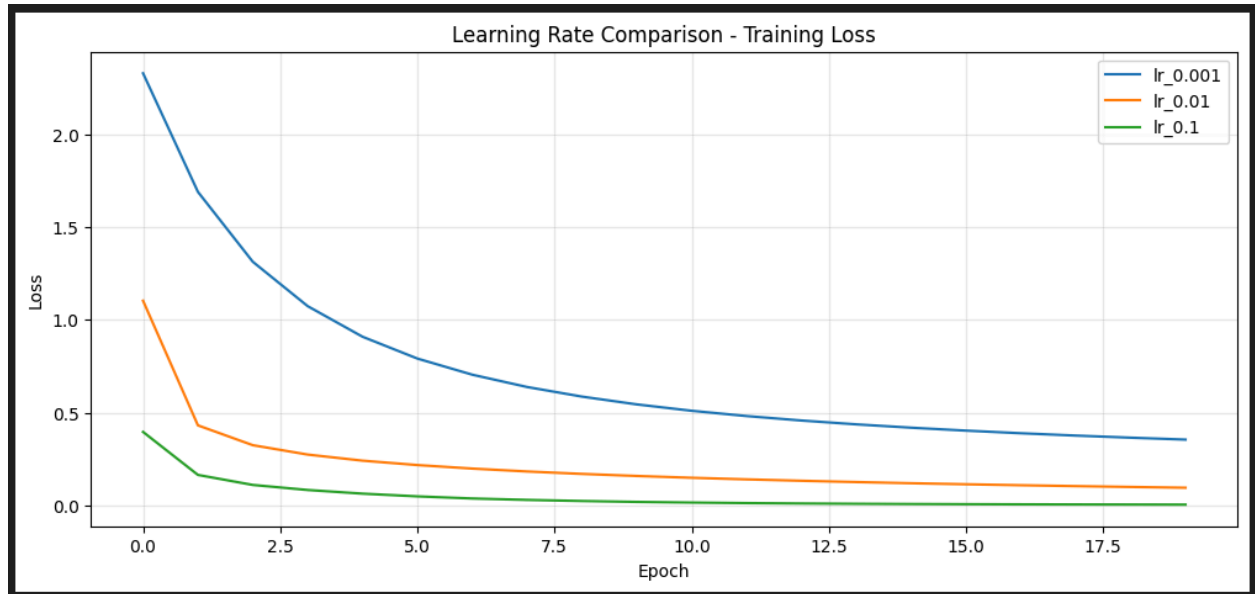
Learning Rate 0.001



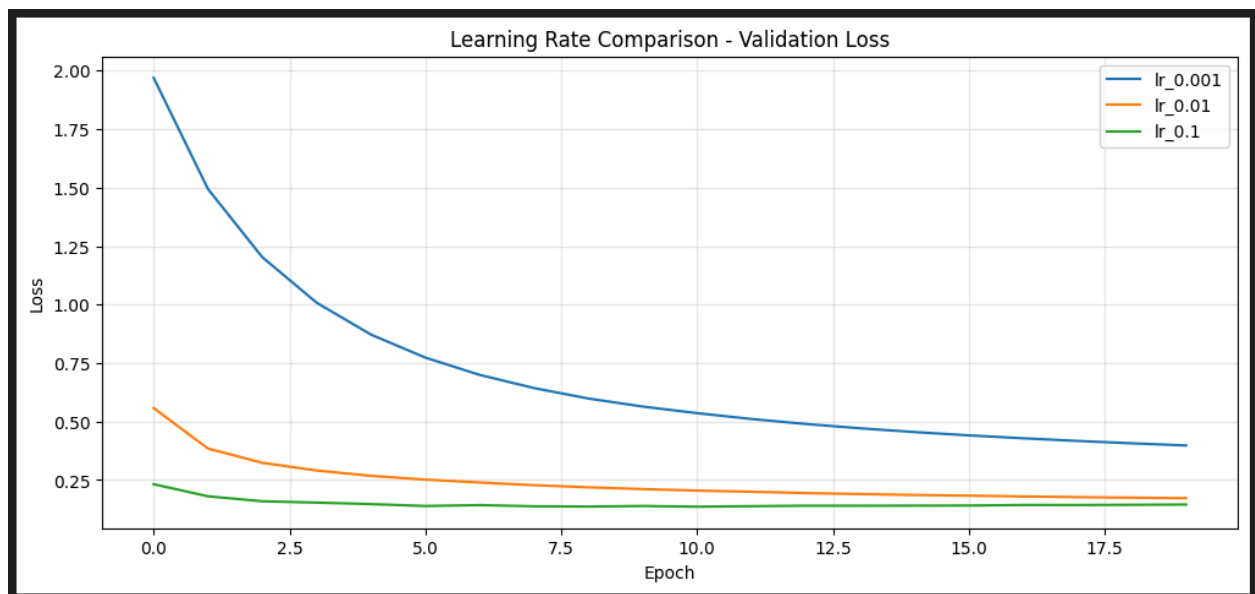
Learning Rate 0.01



Learning Rate 0.1



Training Loss Pengujian Learning Rate



Validation Loss Pengujian Learning Rate

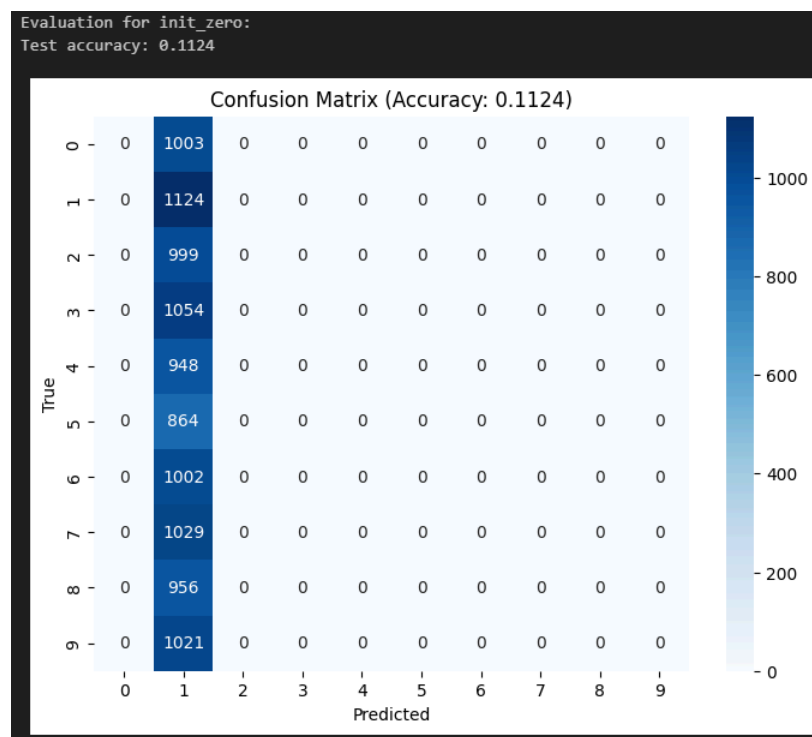
Berdasarkan hasil pengujian learning rate ini didapatkan hasil berikut. Learning rate 0.001 menampilkan konvergensi paling lambat, dengan akurasi tes terendah 0.8925, menunjukkan kemampuan eksplorasi parameter yang sangat lambat. Sebaliknya, learning rate 0.01 memperlihatkan keseimbangan yang baik, dengan akurasi 0.9535, menunjukkan fleksibilitas dalam mengeksplorasi ruang parameter dengan kecepatan dan presisi cukup moderat.

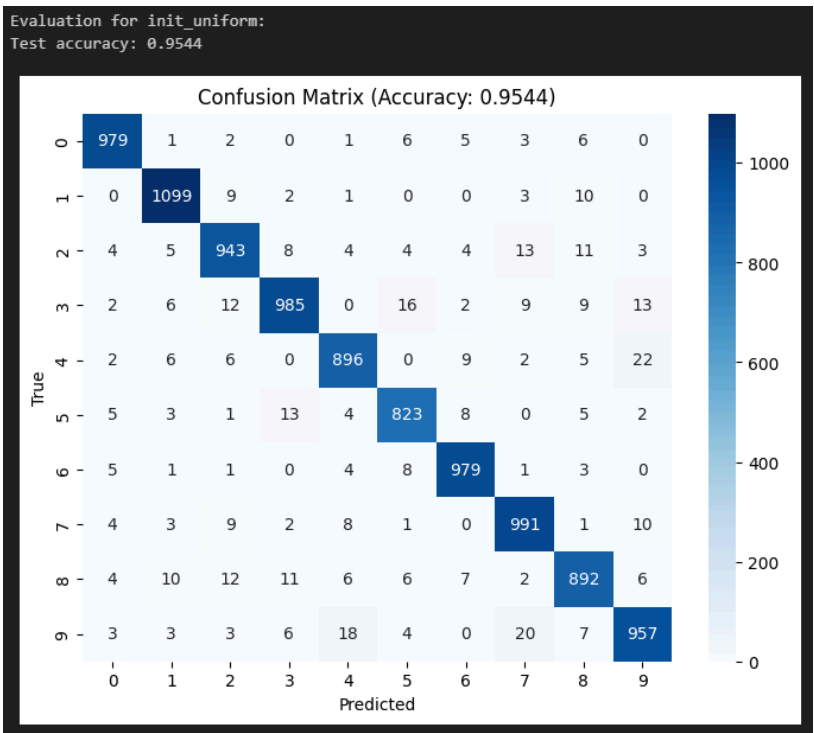
Learning rate 0.1 memiliki hasil terbaik, mencapai akurasi tertinggi 0.9679 dengan kurva loss tercepat dan paling efisien. Hal ini menunjukkan kemampuan jaringan untuk dengan cepat mengadaptasi bobot, menangkap pola kompleks dalam data tanpa terperangkap dalam lokal minimum. Namun, learning rate yang terlalu tinggi berpotensi melewati solusi optimal atau mengalami osilasi dalam ruang pencarian solusi, hal ini juga dapat mengakibatkan exploding gradient problem. Hal ini bisa saja terjadi pada kasus lain, untuk itu sangat baik untuk melakukan testing terlebih dahulu, untuk melakukan checking apakah learning rate akan menimbulkan masalah atau tidak.

2.2.4 Pengaruh Inisialisasi Bobot

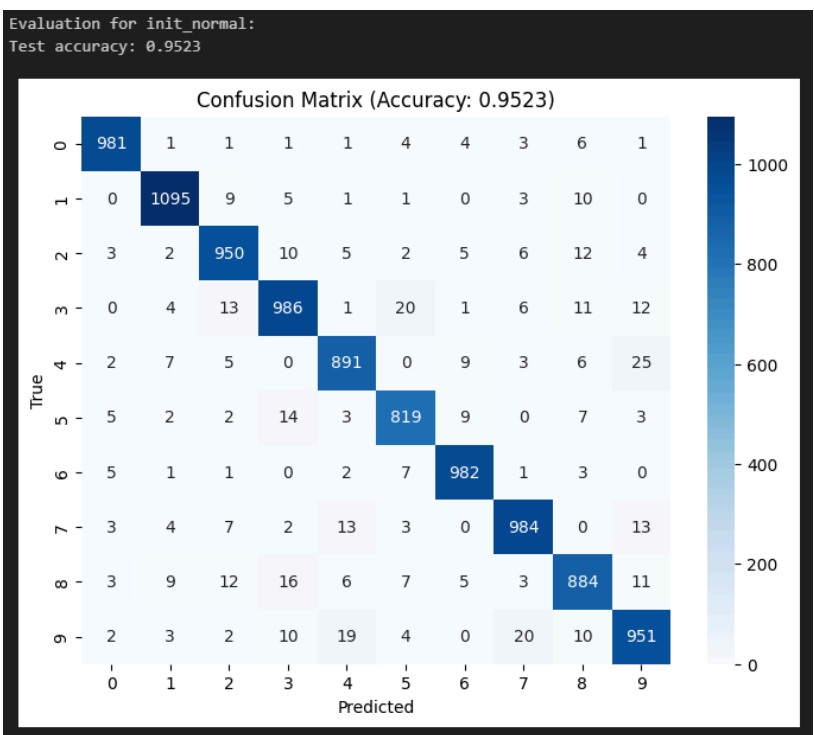
Pengujian pengaruh inisialisasi weight pada model akan dilakukan dengan menggunakan 1 hidden layer dengan **128** neuron. Loss function yang digunakan adalah **categorical cross entropy** loss function dan fungsi aktivasi **ReLU**.

Berikut adalah hasil pengujian pengaruh inisialisasi bobot



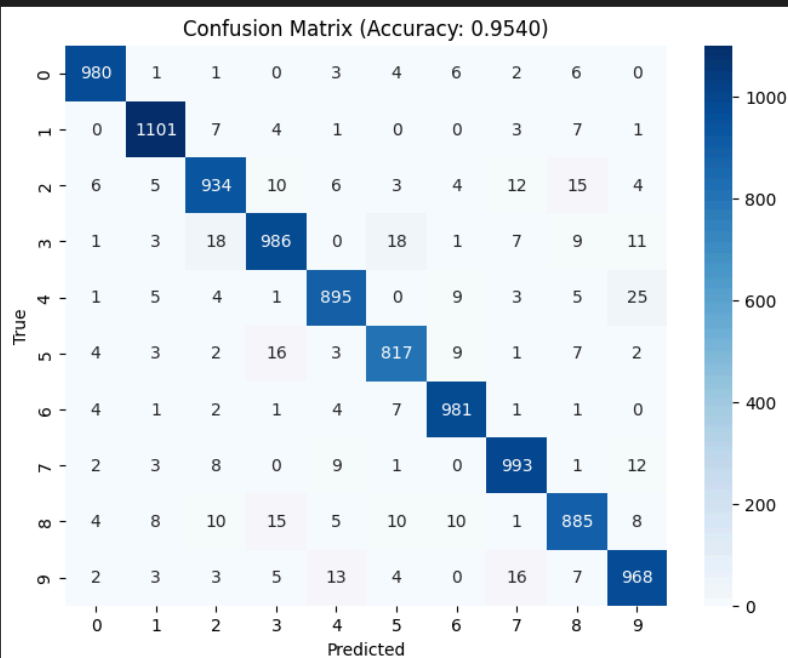


Uniform



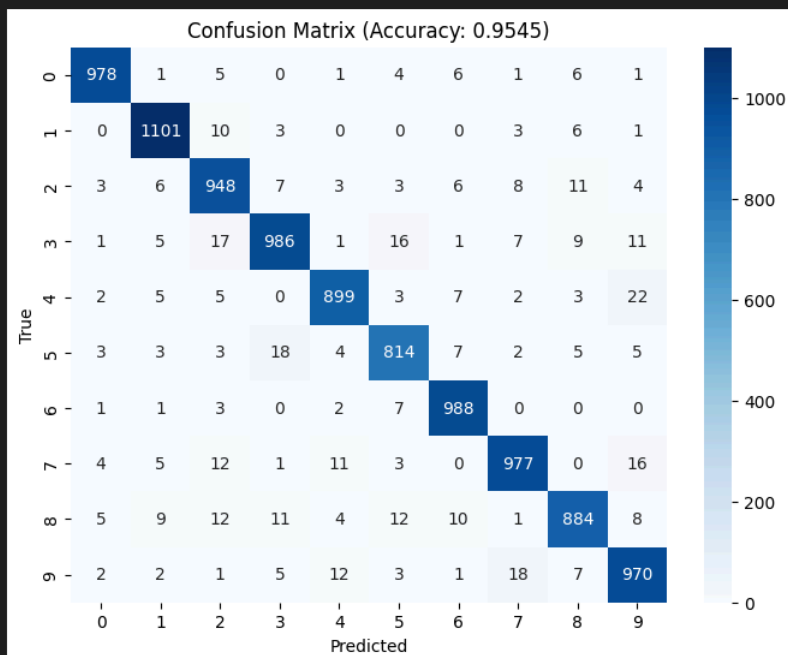
Normal

Evaluation for init_xavier:
Test accuracy: 0.9540

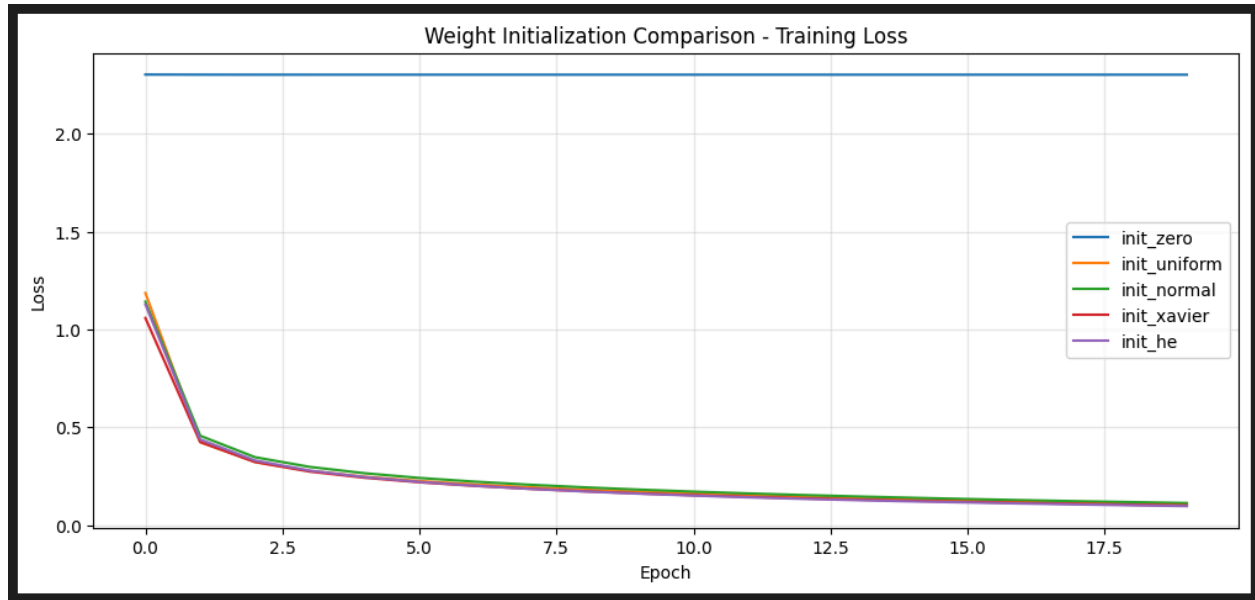


Xavier

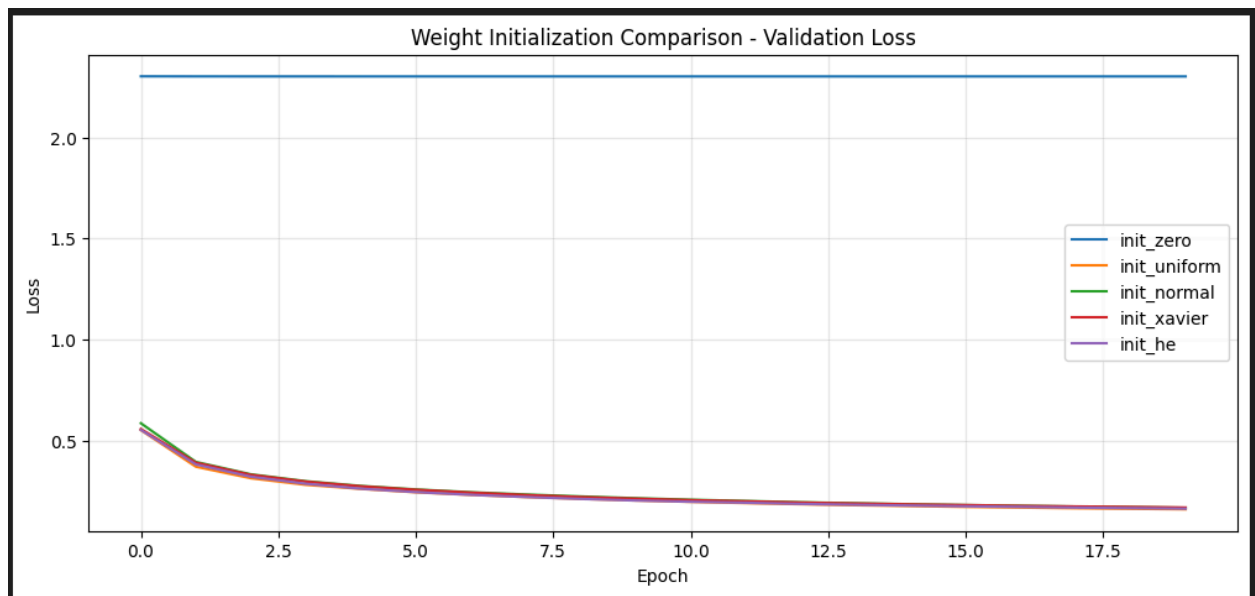
Evaluation for init_he:
Test accuracy: 0.9545



He



Training Loss Pengujian Weight



Validation Loss Pengujian Weight

Dari hasil pengujian di atas, didapatkan beberapa hal. Init zero menunjukkan kinerja terburuk dengan akurasi hanya 0.1124, praktis membuat jaringan gagal untuk melakukan learning karena semua bobot diinisialisasi dengan nol - yang menyebabkan setiap neuron berperilaku identik, sehingga hilangnya kemampuan pembedaan dan diskriminasi jaringan.

Metode uniform (kisaran -0.05 hingga 0.05) dan normal (mean 0, standar deviasi 0.05) menampilkan performa yang jauh lebih baik, dengan akurasi masing-masing 0.9544 dan 0.9523.

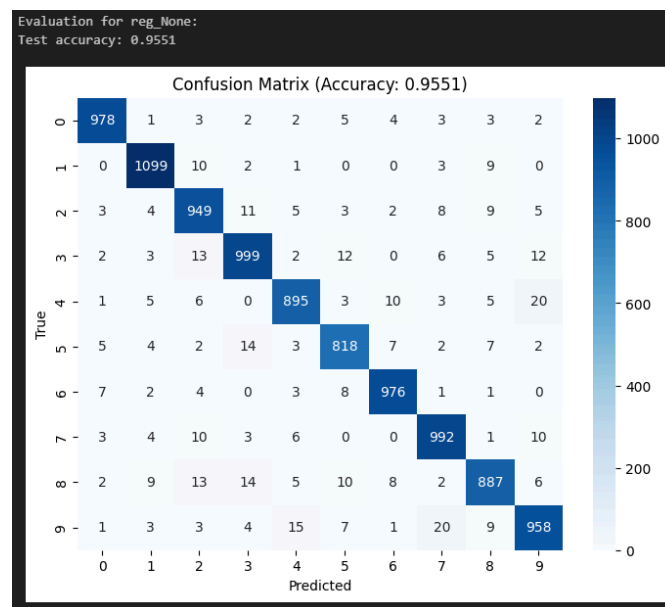
Keduanya memberikan varian bobot awal yang memungkinkan jaringan untuk mulai belajar dengan lebih efektif, karena menghasilkan distribusi bobot yang berbeda di setiap neuron.

Xavier dan He initialization menunjukkan performa yang mirip juga, dengan akurasi masing-masing 0.9540 dan 0.9545. Secara keseluruhan terlihat terdapat 4 inisialisasi yang cukup baik untuk digunakan, tetapi dalam realitanya, Xavier dan He akan dapat lebih baik dibandingkan inisialisasi lainnya, hal ini dikarenakan metode ini dirancang secara khusus untuk menjaga varians sinyal tetap stabil selama proses forward dan backward propagation, yang sangat berguna pada jaringan dalam - mencegah masalah vanishing atau exploding gradient.

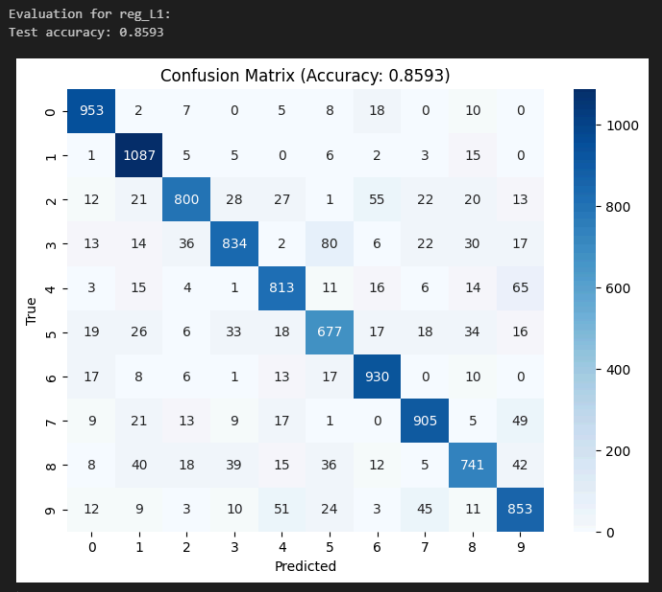
2.2.5 Pengaruh Regularisasi (bonus)

Pengujian pengaruh regularisasi pada model akan dilakukan dengan menggunakan **1** hidden layer dengan **128** neuron. Loss function yang digunakan adalah **categorical cross entropy** loss function dan fungsi aktivasi **ReLU**.

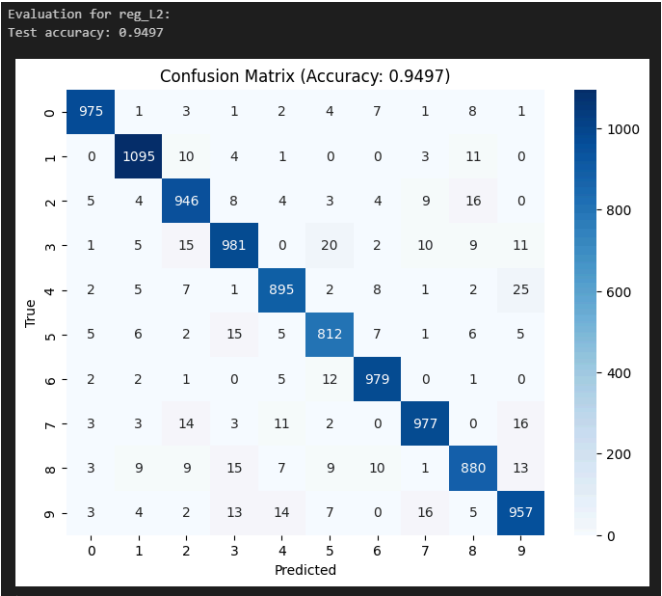
Berikut adalah hasil pengujian pengaruh regularisasi



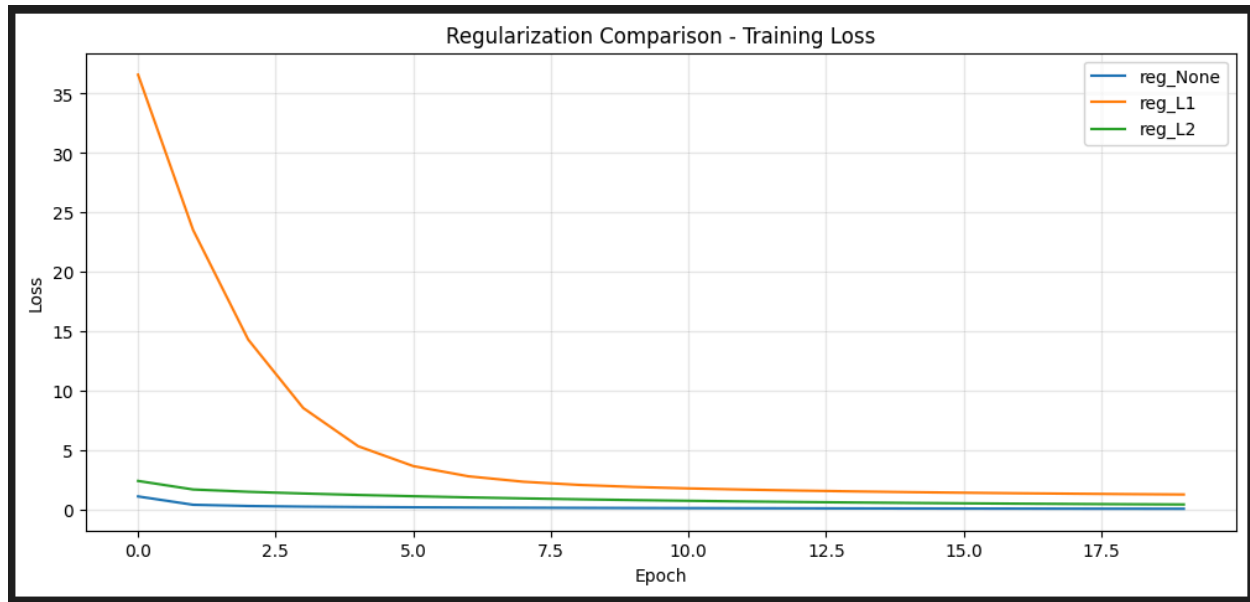
No Reg



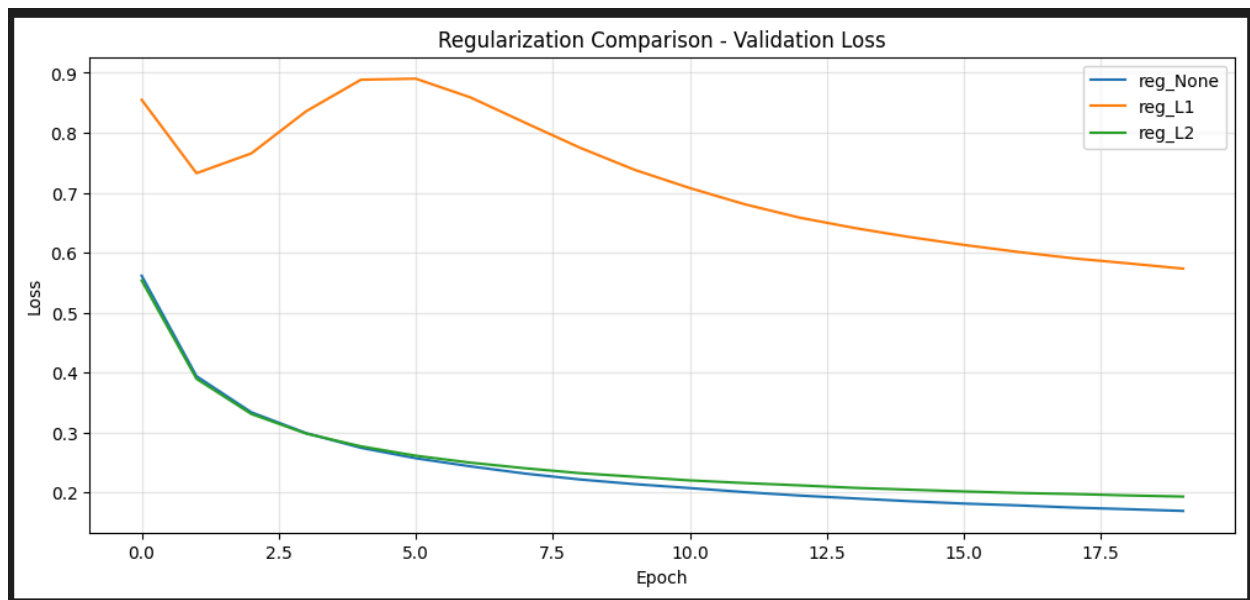
Reg L1t



Reg L2



Training Loss Pengujian Regularisasi



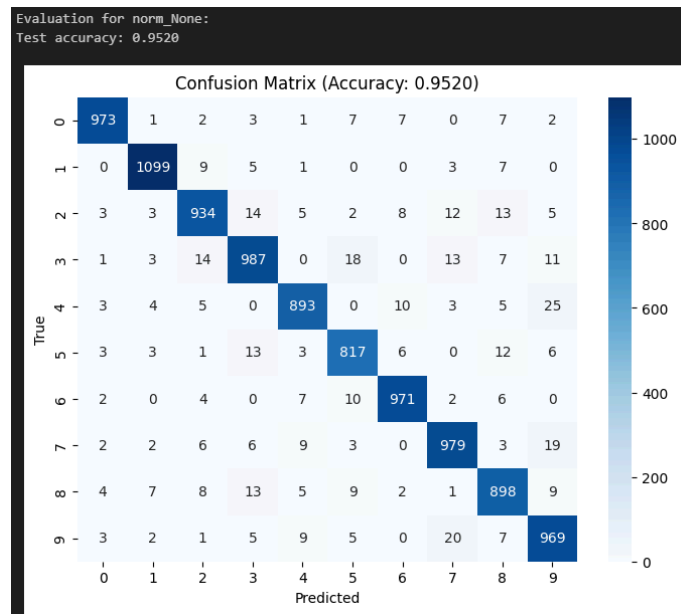
Validation Loss Pengujian Regularisasi

Berdasarkan hasil pengujian di atas, dapat dilihat dengan jelas bahwa L1 menunjukkan pendekatan dengan mendorong kesenjangan pada model. Metode ini in a nutshell melakukan seleksi fitur, sehingga pada kasus ini akurasi menurun hingga 0.8593, menunjukkan hasil yang sangat fluktuatif pada validation. L2 menunjukkan pendekatan yang paling seimbang dengan menambahkan penalti secara kuadrat pada model, L2 menghasilkan hasil yang lebih robust. Sedangkan tidak menggunakan regularisasi sama sekali dapat menghasilkan hasil yang overfit.

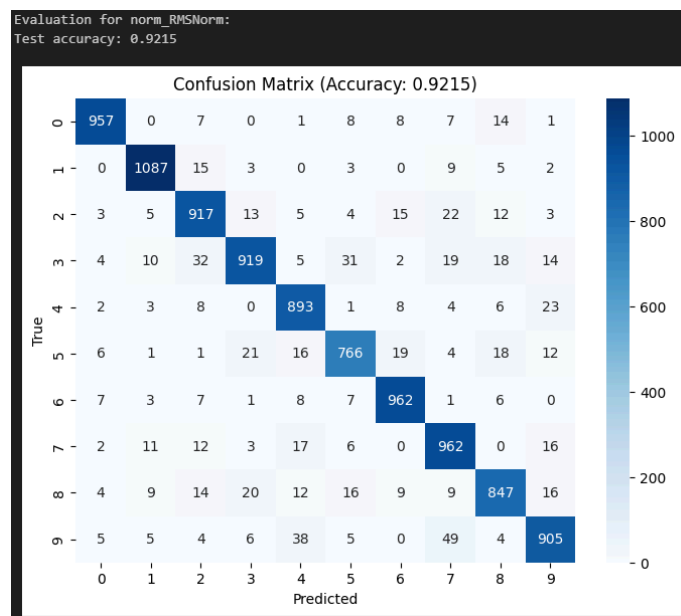
2.2.6 Pengaruh Normalisasi RMSNorm (bonus)

Pengujian pengaruh normalisasi pada model akan dilakukan dengan menggunakan 4 hidden layer dengan **128** neuron tiap layer-nya. Loss function yang digunakan adalah **categorical cross entropy** loss function dan fungsi aktivasi **ReLU**.

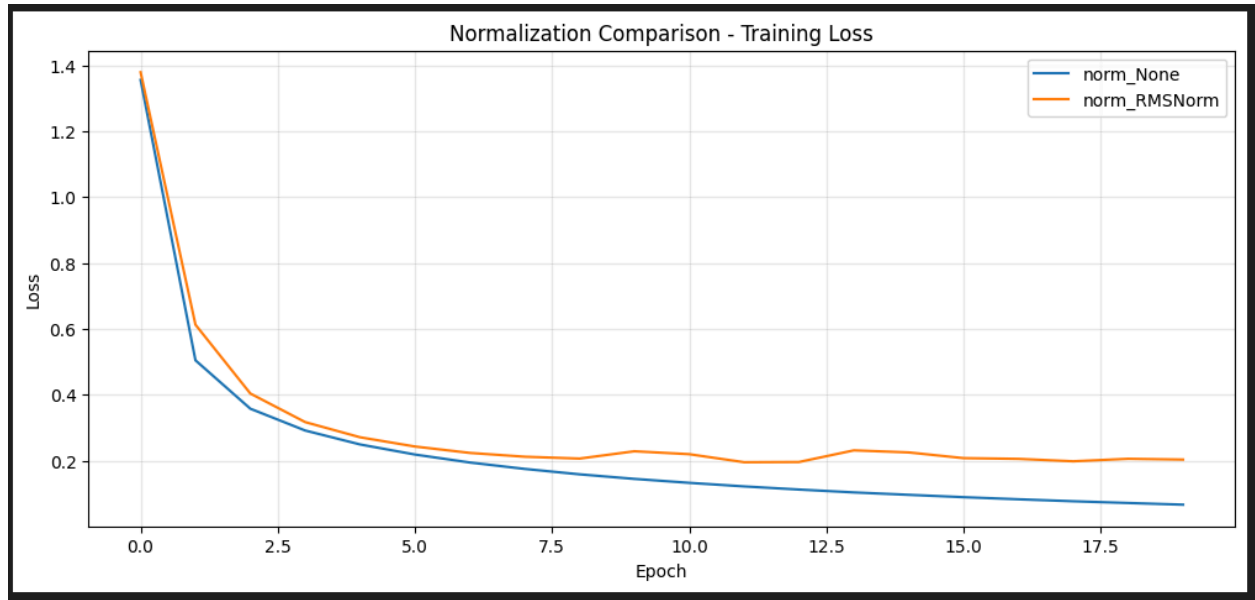
Berikut adalah hasil perbandingannya



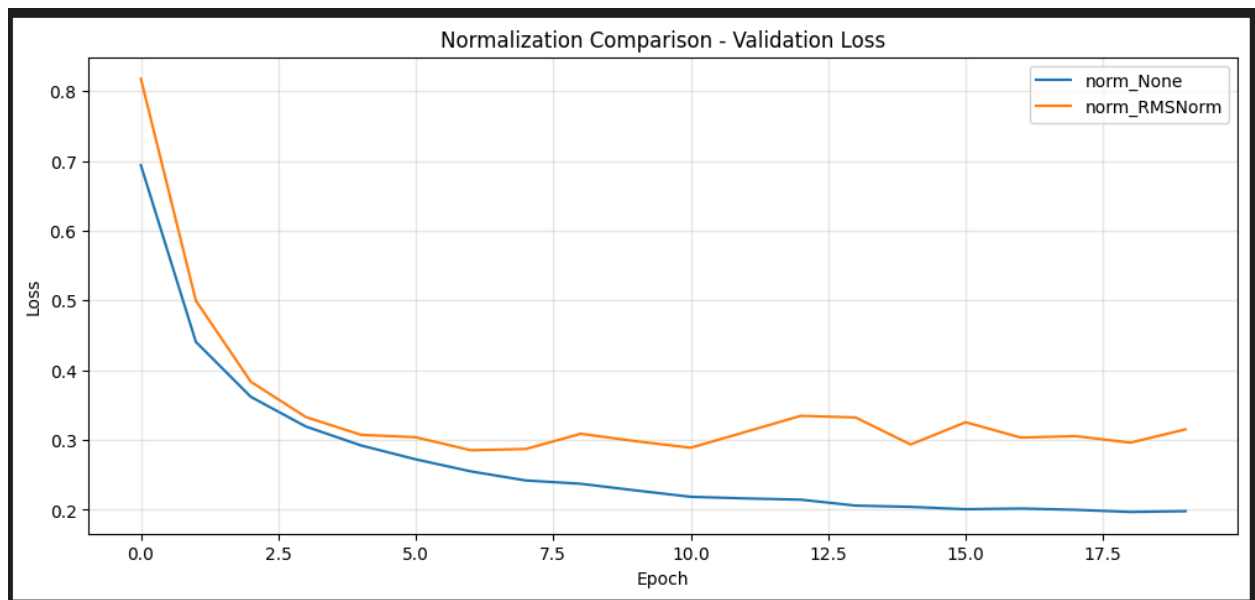
No RMSNorm



Menggunakan RMSNorm



Training Loss Pengujian RMSNorm



Validation Loss Pengujian RMSNorm

Berdasarkan hasil di atas dapat dilihat bahwa tanpa menggunakan normalisasi model mencapai akurasi 0.9520 dengan distribusi prediksi yang relatif konsisten. Sebaliknya, RMSNorm menghasilkan akurasi sedikit lebih rendah pada 0.9215, dengan sebaran kesalahan klasifikasi yang lebih tersebar.

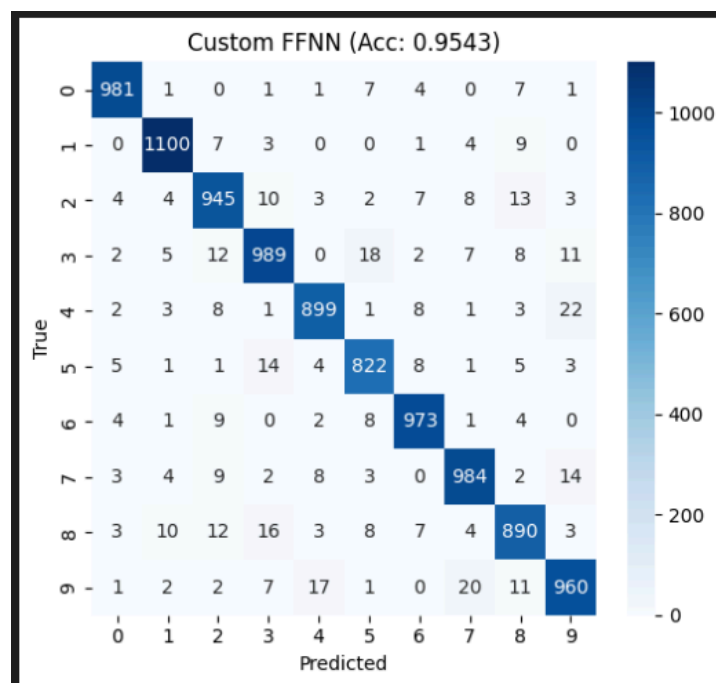
RMSNorm sebenarnya dirancang untuk mengatasi vanishing gradient dan stabilkan proses training, namun dalam kasus spesifik ini tampaknya kurang efektif. Namun dalam kasus ini ada

beberapa hal yang memungkinkan hasil yang sedikit lebih buruk. Diantaranya adalah kemungkinan kompleksitas normalisasi yang mengganggu alur dari gradient descent. Pada praktiknya tidak semua metode normalisasi bersifat universal, efektivitas penerapan RMSNorm sangat tergantung pada kompleksitas dataset, arsitektur dari sebuah jaringan spesifik dan karakter distribusi dari data. Kurva loss menguatkan temuan ini dengan menunjukkan RMSNorm mengalami fluktuasi lebih tinggi, mengindikasikan proses learning yang kurang stabil dibandingkan normalisasi standar.

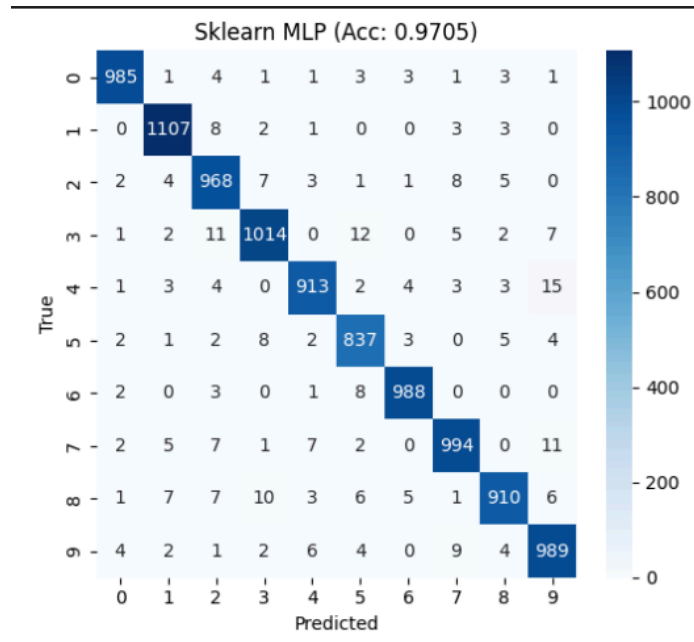
2.2.7 Perbandingan dengan library sklearn

Pengujian perbandingan dengan library sklearn model akan dilakukan dengan menggunakan 1 hidden layer dengan **128** neuron. Loss function yang digunakan adalah **categorical cross entropy** loss function, fungsi aktivasi **ReLU**, Inisialisasi bobot **he**, learning rate **0.01** dan **20** epoch.

Berikut adalah hasil perbandingannya.



Hasil dari implementasi FFNN



Hasil dari sklearn MLP Classifier

Berdasarkan hasil dari kedua model ini dapat dikatakan kedua model menunjukkan kinerja yang sangat baik, dengan akurasi Custom FFNN 0.9543 dan Scikit-learn MLP 0.9705. Perbedaan tipis ini mengindikasikan desain arsitektur dan implementasi yang cukup solid pada implementasi FFNN kelompok kami. Namun model dari scikit learn tetaplah lebih unggul, hal ini wajar dan dapat disebabkan oleh beberapa faktor, diantaranya adalah berikut:

- Implementasi optimasi bawaan yang lebih canggih
Scikit Learn memiliki algoritma optimasi bawaan yang jelas telah teruji, lebih lagi, scikit learn menggunakan advanced gradient descent dan adaptive learning. Walaupun kami mengimplementasikan penurunan learning rate, tetapi hanya dengan melakukan multiplier untuk tiap epoch, sedangkan scikit learn melakukan penyesuaian learning rate secara dinamis selama proses training.
- Algoritma backpropagation yang teruji
Implementasi backpropagation scikit learn sudah melalui tahap uji yang sangat baik. Penanganan fenomena seperti vanishing dan exploding gradient serta gradient clipping sudah jauh lebih advanced dari penanganan yang kami lakukan.
- Preprocessing dan normalisasi bawaan yang efektif
Sklearn secara otomatis melakukan normalisasi, standarisasi, dan scaling yang cocok dan hypertuned untuk model MLPClassifier mereka secara otomatis dan preprocessing yang konsisten sehingga dapat meningkatkan performa model.

Berdasarkan hal ini, perbedaan akurasi sebesar 0.0162 dapat dikatakan sangat minimal, sehingga dapat dikatakan model implementasi FFNN kami cukup memuaskan dan menunjukkan keberhasilan implementasi dari awal.

Kesimpulan dan Saran

Hasil implementasi dalam tugas besar ini melibatkan banyak eksplorasi - eksplorasi dari berbagai aspek kritis dalam FFNN. Setiap eksperimen memberikan hasil yang membantu memahami pengaruh faktor - faktor terhadap model.

Berdasarkan hasil penelitian ini, didapatkan bahwa tidak terdapat pendekatan yang 100% sempurna. Terdapat beberapa parameter yang terbukti terbaik dalam kasus ini. ReLU sebagai fungsi aktivasi paling efektif, serta mengatasi vanishing gradient secara efisien. Inisialisasi Xavier dan He menunjukkan pentingnya inisialisasi bobot yang baik untuk perkembangan learning model. Serta learning rate 0.1 membuktikan keunggulannya dalam mengeksplorasi ruang parameter secara cepat dan presisi. Walau menjadi yang terbaik dalam kasus ini, faktor - faktor ini memiliki kelemahannya masing - masing, seperti bentuk pengaruh dari noise, bentuk dataset dan lain - lain. Learning rate 0.1 dapat menimbulkan explosion gradient, atau fungsi aktivasi sigmoid yang mungkin dapat menjadi hasil yang terbaik jika klasifikasi yang dilakukan adalah klasifikasi biner.

Regularisasi L2 dan normalisasi standar menunjukkan keunggulan dalam mencegah overfitting dan menstabilkan proses training. Perbandingan dengan implementasi sklearn menggarisbawahi bahwa model implementasi FFNN kami dapat mencapai performa yang kompetitif, yang mengindikasikan kualitas rancangan arsitektur dan pemahaman mendalam tentang mekanisme neural network.

Secara fundamental implementasi FFNN memerlukan pertimbangan - pertimbangan cermat, eksperimentasi yang banyak dan sistematis, dan pemahaman komprehensif mengenai interaksi kompleks antara komponen dan parameter model. Tidak terdapat solusi yang instan, melainkan adanya pendekatan iteratif.

Pembagian Tugas

NIM	Nama	Task
13522029	Ignatius Jhon Hezkiel Chan	FFNN, forward, backward, softmax, desc implementasi, dokumen
13522043	Daniel Mulia Putra Manurung	Setup, Activation, loss, initializer, Experiment, dokumen
13522093	Matthew Vladimir Hutabarat	Save, Load, Plot model, testing, dokumen

Referensi

Prince, S. J. (2023). *Understanding deep learning*. MIT Press.

Kunapuli, G. (2023). *Ensemble methods for machine learning*. Simon and Schuster.

Mitchell, T. (1997). *Machine learning*. McGraw-Hill.